

project_first_code

October 21, 2025

```
[ ]: # data analysis
import pandas as pd
import numpy as np
import os
import time
import datetime
import psutil
from cpuinfo import get_cpu_info
import platform
import subprocess
from math import pi
from functools import partial
from sklearn.feature_selection import mutual_info_classif
from scipy.stats import chi2_contingency
from itertools import combinations
from scipy.stats import skew
from sklearn.preprocessing import PowerTransformer
from imblearn.over_sampling import SMOTENC
from sklearn.preprocessing import MinMaxScaler
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from pycaret.classification import *
from sklearn.model_selection import learning_curve
from sklearn.metrics import roc_curve, auc
# data visualization
import matplotlib.pyplot as plt
from matplotlib import rcParams
from matplotlib.table import Table
import seaborn as sns
from sklearn.metrics import confusion_matrix, roc_auc_score, \
    ConfusionMatrixDisplay, auc, roc_curve
from sklearn.metrics import cohen_kappa_score, f1_score, precision_score, \
    recall_score, accuracy_score, matthews_corrcoef
from sklearn.model_selection import learning_curve
from sklearn.ensemble import GradientBoostingClassifier, \
    RandomForestClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier, ExtraTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
```

```

from xgboost import XGBRFClassifier
from lightgbm import LGBMClassifier
from sklearn.ensemble import VotingClassifier, StackingClassifier
#warnings
import warnings, traceback
warnings.filterwarnings("ignore")
from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

```

[ ]: file_path = "/content/drive/MyDrive/project_first/
      ↪clean_sql_export_for_datamining_day_to_quartile.xlsx"
dataset = pd.read_excel(file_path,sheet_name="data")
pd.set_option("display.max_columns",None) # for showing all columns
pd.set_option("display.expand_frame_repr",False) # to avoid data clutter
dataset

```

```

[ ]:
      serial  zsituation  brithkind  sex_b  weight_b  zcomment
joutcomment  cout_joutcomment  drylendays  prelendays  DFS_milkperiod  DFS
dis_ab  count_ab              tmom              father  age_breeding  day_tdate
mount_tdate  year_tdate  MilkPeriod  tkind  SpermNum  PersonCode  dim_tdate
avgmilk  faz_mah  time_faz  not_return
0      1001369063103130      2      1      f      41      2
1      9      54      273      94  449      .
0  1001366110402530  1775836      163.409836      forth      3
1425      10      2  120173998      1011      339      19.6      21
2      0
1      1001369063103130      2      1      f      41      2
1      9      54      273      94  449      .
0  1001366110402530  1775836      164.131147      tirth      4
1425      10      2  124717307      1011      361      17.2      2
2      0
2      1001369071303650      1      1      m      .      2
1      8      108      265      47  484      .
0  1001366032200610  1775836      166.754098      tirth      7
1425      11      2  17256591      1011      47      27.4      21
2      0
3      1001369071303650      1      1      m      .      2
1      8      108      265      47  484      .
0  1001366032200610  1775836      168.000000      first      9
1425      11      2  126365823      1010      85      23.5      1
2      0
4      1001369071303650      1      1      m      .      2
1      8      108      265      47  484      .
0  1001366032200610  1775836      168.557377      tirth      9
1425      11      2      9043859      1011      102      20.2      2

```

```

2          0
...
...
...
...
...
...
118991  1761386102005300          2          1    f          44          2
0          0          95          279          71 445          .
0  1761381072004850  176138405040444          85.836065          tirth          5
1436          5          2  62916235          1028          248 23.25          21
4          0
118992  1761386102005300          2          1    f          44          2
0          0          95          279          71 445          .
0  1761381072004850  176138405040444          87.409836          second          7
1436          5          2  69791579          1028          296 25.1          11
2          0
118993  1761386102205370          2          1    m          48          1
0          0          0          279          64 437          .
0  1761383110206710  128600717          28.163934          first          6
1431          1          2  129008732          1028          64 34.8          11
4          1
118994  1761386102205370          2          1    f          40          1
0          0          58          280          49 437          .
0  1761383110206710  128600717          38.950819          second          5
1432          2          2  66011448          1028          49 59.9          11
2          1
118995  1761386102205370          2          1    f          40          1
0          0          58          280          49 437          42
1  1761383110206710  128600717          48.590163          second          3
1433          2          2  61546915          1034          343          .          11
2          0

```

[118996 rows x 29 columns]

```
[ ]: dataset_copy = dataset.copy() #a copy of dataset has made to make sure that
      ↪ main dataset doesnot change
```

```
[ ]: dataset_copy.info() #complete information of dataset's columns has printed that
      ↪ contains column name, number of data, and data type
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 118996 entries, 0 to 118995
Data columns (total 29 columns):
#   Column          Non-Null Count  Dtype
---  -
0   serial          118996 non-null int64
1   zsituation      118996 non-null int64
2   brithkind       118996 non-null int64

```

```

3  sex_b          118996 non-null object
4  weight_b       118996 non-null object
5  zcomment       118996 non-null object
6  joutcomment    118996 non-null object
7  cout_joutcomment 118996 non-null int64
8  drylendays     118996 non-null int64
9  prelendays     118996 non-null int64
10 DFS_milkperiod 118996 non-null int64
11 DFS            118996 non-null int64
12 dis_ab         118996 non-null object
13 count_ab       118996 non-null int64
14 tmom           118996 non-null object
15 father         118996 non-null object
16 age_breeding   118996 non-null float64
17 day_tdate      118996 non-null object
18 mount_tdate    118996 non-null int64
19 year_tdate     118996 non-null int64
20 MilkPeriod     118996 non-null int64
21 tkind          118996 non-null int64
22 SpermNum       118996 non-null object
23 PersonCode     118996 non-null object
24 dim_tdate      118996 non-null int64
25 avgmilk        118996 non-null object
26 faz_mah        118996 non-null int64
27 time_faz       118996 non-null int64
28 not_return     118996 non-null int64
dtypes: float64(1), int64(17), object(11)
memory usage: 26.3+ MB

```

```

[ ]: dataset_copy.rename(columns={'serial':'Animal', 'zsituation':'PAS', 'brithkind':
↳ 'BT',
                                'sex_b':'SEX', 'weight_b':'WB', 'zcomment':'DS',
↳ 'joutcomment':'RP',
                                'cout_joutcomment':'TNR', 'drylendays':'DR',
↳ 'prelendays': 'GL',
                                'DFS_milkperiod':'DMP', 'DFS':'DFS', 'dis_ab':
↳ 'IFI',
                                'count_ab':'NPA', 'tmom':'DIM', 'father':'SIR',
↳ 'age_breeding':'AI',
                                'day_tdate':'DI', 'mount_tdate':'MI', 'year_tdate':
↳ 'YI',
                                'MilkPeriod':'LP', 'tkind':'TP', 'SpermNum':'S',
↳ 'PersonCode':'AIT',
                                'dim_tdate':'DM', 'avgmilk':'AM', 'faz_mah':'CL',
↳ 'time_faz':'CI',
                                'not_return':'PS'}, inplace=True)#columns names
↳ has changed to simple names so they can be used easily in other stages

```

```
[ ]: dataset_copy
```

```
[ ]:
      Animal PAS BT SEX WB DS RP TNR DR GL DMP DFS IFI
      NPA      DIM      SIR      AI      DI MI      YI LP TP
      S AIT DM AM CL CI PS
0      1001369063103130 2 1 f 41 2 1 9 54 273 94 449 .
0 1001366110402530 1775836 163.409836 forth 3 1425 10 2
120173998 1011 339 19.6 21 2 0
1      1001369063103130 2 1 f 41 2 1 9 54 273 94 449 .
0 1001366110402530 1775836 164.131147 tirth 4 1425 10 2
124717307 1011 361 17.2 2 2 0
2      1001369071303650 1 1 m . 2 1 8 108 265 47 484 .
0 1001366032200610 1775836 166.754098 tirth 7 1425 11 2
17256591 1011 47 27.4 21 2 0
3      1001369071303650 1 1 m . 2 1 8 108 265 47 484 .
0 1001366032200610 1775836 168.000000 first 9 1425 11 2
126365823 1010 85 23.5 1 2 0
4      1001369071303650 1 1 m . 2 1 8 108 265 47 484 .
0 1001366032200610 1775836 168.557377 tirth 9 1425 11 2
9043859 1011 102 20.2 2 2 0
...
...
...
118991 1761386102005300 2 1 f 44 2 0 0 95 279 71 445 .
0 1761381072004850 176138405040444 85.836065 tirth 5 1436 5 2
62916235 1028 248 23.25 21 4 0
118992 1761386102005300 2 1 f 44 2 0 0 95 279 71 445 .
0 1761381072004850 176138405040444 87.409836 second 7 1436 5 2
69791579 1028 296 25.1 11 2 0
118993 1761386102205370 2 1 m 48 1 0 0 0 279 64 437 .
0 1761383110206710 128600717 28.163934 first 6 1431 1 2
129008732 1028 64 34.8 11 4 1
118994 1761386102205370 2 1 f 40 1 0 0 58 280 49 437 .
0 1761383110206710 128600717 38.950819 second 5 1432 2 2
66011448 1028 49 59.9 11 2 1
118995 1761386102205370 2 1 f 40 1 0 0 58 280 49 437 42
1 1761383110206710 128600717 48.590163 second 3 1433 2 2
61546915 1034 343 . 11 2 0
```

```
[118996 rows x 29 columns]
```

```
[ ]: for column in dataset_copy.columns:#data of contiuous columns and categories of
      ↪categorical columns have printed
      print(f"{column}")
      print(dataset_copy[column].unique())
      print("\n")
```

```
Animal
```

[1001369063103130 1001369071303650 1001369102208260 ... 1761386101805250
1761386102005300 1761386102205370]

PAS

[2 1 12]

BT

[1 2 3]

SEX

['f' 'm' '2m' '2f' '2' '9']

WB

['41' '.' '40' '45' '46' '38' '44' '43' '29' '35' '48' '47' '50' '49' '52'
'60' '51' '42' '39' '37' '53' '55' '77' '63' '74' '62' '58' '34' '33'
'81' '36' '57' '30' '66' '61' '67' '76' '56' '28' '64' '4' '26' '32' '18'
'78' '31' '23' '88' '73' '54' '59' '27' '13' '3.6' '21' '84' '783' '22'
'70' '12' '69' '75' '82' '38.8' '72' '80' '79' '24' '4.4' '68' '71' '90'
'65' '85' '20' '91' '25' '11' '0' '16' '38.1' '273' '249' '307' '234'
'87' '3.7' '19' '83' '3.2' '286' '2.7' '15' '93' '2.9' '413' '86' '10'
'92' '13.4' '38.9' '17' '38.2' '39.5' '40.2' '2' '630' '38.6' '637' '121'
'145' '138' '115' '111' '74.3' '120' '7' '3.1' '44.5' '122' '1.47' '160'
'135' '145.5' '119' '142' '365' '3.4' '98.5' '118' '1.42' '47.75' '10.2'
'38.7' '14' '662' '160.4' '454' '139' '39.2' '358' '174' '38.3' '131.5'
'39.1' '144.5' '1.45' '114' '1.4' '671' '455' '96.1' '1' '129.5' '485'
'112' '575' '155.4' '51.5' '38.5' '174.4' '89' '168' '110' '256' '82.75'
'44.4' '108' '43.5' '138.6' '126.8' '137' '131.6' '126.6' '483' '115.6'
'47.5' '207' '163.4' '40.7' '148' '127.6' '82.4' '125.3' '57.9' '3']

DS

['2' '1' '3' '4' '.']

RP

['1' '0' '.']

TNR

[9 8 7 10 6 4 5 2 1 3 0]

DR

[54 108 111 85 57 119 115 79 364 132 60 182 36 87 76 72 167 64

62 70 68 42 56 61 63 52 67 58 69 65 66 73 100 230 55 160
 128 77 98 75 139 137 105 102 71 241 46 50 141 168 135 121 122 90
 106 59 210 109 206 92 237 157 95 81 347 53 124 104 21 154 202 153
 172 180 189 107 197 88 261 78 103 162 125 127 149 265 34 146 49 44
 143 101 118 37 89 221 41 243 152 136 9 193 51 232 16 96 97 48
 133 173 134 83 47 144 234 99 19 91 1 223 165 148 250 219 246 181
 30 74 28 82 86 142 145 273 138 155 15 117 114 228 211 84 245 94
 32 24 33 112 208 203 40 43 271 130 176 150 126 123 45 38 187 5
 4 171 22 80 31 93 175 215 116 129 18 217 260 29 201 151 140 244
 7 205 249 170 35 179 8 169 213 218 39 113 231 192 204 226 159 267
 110 195 161 164 131 183 214 252 216 199 185 282 147 313 262 156 174 27
 158 235 177 0 259 251 212 184 272 25 120 253 11 321 186 6 224 20
 163 26 23 188 14 236 2 198 229 286 318 10 354 207 178 389 288 255
 17 190 191 263 375 196 293 248 225 289 247 194 13 350 575 281 239 166
 426 3 220 256 266 242 12 227 238 200 632 222 233 275 359 209 307 240
 336 504]

GL

[273 265 281 282 275 278 283 279 280 290 255 249 276 277 285 262 287 286
 272 270 268 274 288 284 289 266 271 264 240 259 263 291 267 269 229 231
 256 29 243 252 254 292 230 253 246 293 294 258 210 223 187 239 261 295
 248 257 247 209 238 297 237 251 373 260 312 244 241 228 225 221 232 234
 233 250 299 219 298 227 302 245 212 242 318 226 43 304 224 296 308 236
 88 143 153 202 216 320 309 235 129 193 310 300 218 208 301 205 307 303
 220 213 222 311 305 194 104 203 172 195 151 149 333 306 160 154]

DMP

[94 47 74 79 46 77 91 67 58 128 42 63 172 44 65 75 99 106
 114 60 113 83 78 122 57 50 28 51 53 101 200 71 45 69 55 144
 66 110 93 82 49 111 173 54 90 73 102 76 52 87 43 183 40 85
 103 81 80 64 48 134 61 56 119 170 166 88 123 127 70 105 118 72
 41 135 130 100 89 39 104 92 95 107 38 59 139 236 68 136 117 108
 151 84 140 124 62 86 96 126 286 162 37 120 129 34 116 131 36 191
 112 182 97 192 32 115 121 132 158 185 472 141 30 275 33 98 109 31
 226 153 163 230 133 35 148 212 150 29 161 169 186 152 175 285 155 171
 149 159 125 165 176 413 328 245 376 216 206 235 142 179 189 219 157 174
 208 147 210 234 246 154 156 168 19 190 160 138 187 188 221 137 227 145
 253 164 461 194 146 24 143 207 316 -43 204 26 180 184 178 220 224 202
 181 167 257 197 306 217 293 422 209 225 349 205 27 671 222 196 249 177
 321 498 198 218 273 256 259 343 295 201 215 242 276 14 258 281 195 232
 298 290 213 269 228 266 338 251 303 203 243 250 199 231 370 23 241 193
 291 233 369 238 260 244 264 482 354 240 25 336 13]

DFS

[449 484 473 485 479 499 510 482 471 475 501 495 498 532

529	497	504	487	491	500	493	486	490	492	494	513	496	506
508	488	511	489	503	519	502	507	700	552	524	535	452	509
1120	1153	457	542	520	610	512	538	523	531	518	517	514	527
423	553	391	534	521	526	515	516	541	543	528	525	546	560
522	533	536	530	602	609	612	603	572	569	554	599	565	588
555	505	623	559	583	557	564	540	585	558	549	551	562	459
548	580	537	483	341	561	556	589	462	291	576	481	469	476
467	478	461	456	454	474	460	464	458	470	466	455	465	463
453	468	293	448	472	447	444	439	442	450	433	446	445	451
432	429	436	440	426	420	441	430	443	431	435	438	437	428
427	434	608	422	409	402	397	395	393	425	480	477	278	421
416	412	414	655	539	410	581	545	396	403	360	367	424	593
665	573	574	571	566	605	384	639	418	417	419	415	563	669
763	627	413	670	570	392	577	405	411	407	408	406	343	386
586	747	575	201	200	221	404	332	401	400	390	398	383	890
807	889	803	844	787	843	786	399	394	370	388	579	594	568
550	366	544	376	387	378	78	375	380	347	374	377	389	381
354	385	382	379	371	76	373	372	369	365	368	364	344	254

633]

IFI

[' ' '41' '61' '102' '69' '90' '113' '53' '58' '24' '30' '56' '40' '97'
'21' '16' '47' '77' '26' '83' '25' '125' '13' '62' '70' '143' '38' '63'
'84' '109' '111' '18' '65' '48' '34' '93' '115' '173' '193' '23' '46'
'27' '79' '59' '107' '131' '151' '45' '36' '95' '121' '32' '98' '33'
'100' '85' '20' '141' '184' '74' '31' '52' '10' '44' '105' '42' '92' '35'
'49' '72' '94' '117' '138' '194' '81' '78' '146' '51' '28' '89' '39' '91'
'14' '54' '50' '55' '86' '119' '110' '135' '37' '88' '171' '191' '73'
'11' '12' '104' '132' '174' '75' '156' '187' '64' '80' '161' '164' '210'
'253' '275' '101' '130' '149' '168' '239' '339' '76' '154' '179' '17'
'29' '99' '123' '43' '221' '68' '15' '127' '96' '22' '66' '175' '57'
'152' '214' '60' '128' '71' '158' '235' '265' '114' '206' '103' '212'
'259' '136' '144' '82' '122' '67' '116' '87' '106' '126' '160' '180'
'223' '243' '312' '332' '225' '120' '272' '112' '266' '133' '177' '200'
'157' '181' '202' '354' '108' '137' '176' '150' '172' '198' '226' '227'
'247' '124' '147' '209' '233' '134' '118' '237' '167' '166' '190' '19'
'9' '159' '216' '252' '274' '370' '393' '236' '140' '196' '215' '240'
'263' '129' '182' '230' '298' '1' '162' '207' '242' '269' '4' '142' '293'
'155' '248' '170' '217' '178' '276' '340' '389' '278' '311' '222' '262'
'7' '220' '238' '326' '372' '213' '255' '153' '258' '282' '303' '386'
'412' '435' '459' '199' '208' '295' '318' '139' '362' '365' '394' '145'
'317' '338' '254' '284' '148' '197' '219' '229' '251' '188' '165' '186'
'289' '337' '441' '505' '528' '201' '249' '270' '232' '185' '163' '301'
'348' '169' '323' '277' '300' '325' '349' '374' '406' '183' '228' '5'
'195' '218' '189' '273' '292' '345' '3' '204' '246' '203' '231' '257'
'305' '205' '268' '320' '6' '369' '420']

NPA

[0 1 2 3 4 5]

DIM

['1001366110402530' '1001366032200610' '1001365022600530' ...
'1761382092105990' '1761381072004850' '1761383110206710']

SIR

['1775836	'	'1701583	'	'1786000	'
'1787178	'	'1897009	'	'375606	'
'1920908	'	'1860287	'	'1863878	'
'104166	'	'376099	'	'363162	'
'364859	'	'1988532	'	'103109	'
'362017	'	'103097	'	'1858047	'
'367385	'	'373367	'	'352790	'
'310218418	'	'802788566	'	'990924171	'
'104725	'	'390998	'	'1834219	'
'2016536	'	'313213252	'	'2016954	'
'1847801	'	'2018457	'	'389265	'
'1924058	'	'391790	'	'1975644	'
'1990589	'	'1811342	'	'1993701	'
'390057	'	'1792928	'	'311651443	'
'100087	'	'1966387	'	'1872906	'
'355662	'	'104178	'	'1841232	'
'364475	'	'100812	'	'392457	'
'109644	'	'101925	'	'107741	'
'392405	'	'106884	'	'1879085	'
'394477	'	'383622	'	'401173	'
'382283	'	'990429366	'	'107656	'
'2074847	'	'1969877	'	'990429436	'
'105403	'	'106751	'	'393854	'
'105463	'	'390403	'	'106547	'
'990386775	'	'107365	'	'108964	'
'107238	'	'4487040904	'	'514351924	'
'106856	'	'106536	'	'107108	'
'511633432	'	'109027	'	'376455	'
'108940	'	'369275	'	'5159645	'
'402300	'	'400665	'	'104899	'
'382748	'	'330643	'	'103690	'
'100100	'	'248328	'	'103998	'
'5186003165	'	'2021095	'	'109859	'
'3588001281	'	'108365	'	'389459	'
'2071864	'	'2092215	'	'3586049301	'
'2093675	'	'1929410	'	'109863	'
'2103297	'	'110191	'	'2050777	'

'461785537	' '108982	' '461449107	'
'457324883	' '108706	' '343514	'
'1810969	' '103992	' '1971523	'
'1631223	' '662014	' '399250	'
'110190	' '400935	' '405659	'
'2125280	' '2141196	' '662246	'
'1111111	' '392452	' '5280115	'
'109509	' '347045	' '662016	'
'2161395	' '2138587	' '393207	'
'109463	' '662164	' '662300	'
'2014879	' '113146	' '124435	'
'662163	' '392637	' '662167	'
'109492	' '2184212	' '112897	'
'2113536	' '662017	' '113608	'
'784342509	' '1821208	' '1903173	'
'2129064	' '113145	' '780798304	'
'32362	' '402825	' '2040728	'
'2120405	' '105277	' '106572	'
'2164027	' '5429693	' '5457798	'
'2204207	' '792920438	' '2203292	'
'125880	' '558185	' '5319769	'
'5470579	' '127157	' '105378	'
'2163822	' '17156007	' '776846536	'
'17140707	' '2179916	' '5279989	'
'17187297	' '127152	' '17292144	'
'17039235	' '17215132	' '127390	'
'17243574	' '17115939	' '127310	'
'105371	' '17064716	' '6518623	'
'17131030	' '6476098	' '17173317	'
'2200711	' '6715365	' '127159	'
'6478288	' '2159770	' '2189589	'
'829877881	' '2198026	' '800074	'
'5364343	' '5470562	' '2205082	'
'127460	' '2195662	' '105927	'
'2231562	' '2171645	' '2247758	'
'125676	' '2272964	' '5559822	'
'2253348	' '128001	' '2157256	'
'125674	' '128076	' '2137511	'
'2265005	' '250787	' '6130765	'
'2214963	' '5808350	' '2230733	'
'2229383	' '128075	' '2182318	'
'127654	' '2234121	' '2260889	'
'125671	' '2233328	' '6812661	'
'7057181	' '6483076	' '125321	'
'2209298	' '6026421	' '2150948	'
'125570	' '2245673	' '2291624	'
'2297473	' '2080263	' '2261692	'
'127212	' '2294436	' '2246936	'

'2252058	' '2277777	' '2124357	'
'125312	' '114861	' '2267089	'
'110914	' '2287161	' '2094527	'
'2266008	' '125684	' '115597	'
'2161674	' '2259506	' '2260769	'
'2294096	' '17373198	' '125572	'
'17214653	' '128940927	' '17117045	'
'129099688	' '127617192	' '124920323	'
'17103268	' '125933326	' '125814	'
'17013604	' '207124691	' '2299308	'
'126142613	' '17304779	' '17131025	'
'123586621	' '17099649	' '127221	'
'2299311	' '128332	' '2301129	'
'127431	' '128000	' '17184708	'
'17226843	' '2193272	' '2302172	'
'2212464	' '17098093	' '2260038	'
'2299786	' '17302940	' '2289548	'
'2275578	' '17007935	' '2291369	'
'17012860	' '17184137	' '128331	'
'128327	' '121072478	' '2234744	'
'127407	' '18060938	' '17349617	'
'127438	' '2280875	' '120173998	'
'6831072	' '2269075	' '6754099	'
'124690866	' '121049625	' '126941409	'
'17366195	' '17273387	' '17231270	'
'2272874	' '18040136	' '127442	'
'124717307	' '17365628	' '122185573	'
'124109429	' '17256591	' '125860398	'
'125521956	' '125997863	' '126365823	'
'140459582	' '204342911	' '160325582	'
'9043859	' '6797081	' '17058140	'
'123982137	' '6975632	' '6780958	'
'214694884	' '159659261	' '120754720	'
'233821937	' '2290597	' '123645630	'
'17151586	' '125178587	' '128468	'
'128501	' '128520	' '128504	'
'17342837	' '128502	' '128330	'
'128835	' '128368028	' '6682654	'
'17089950	' '2259250	' '128510	'
'120614769	' '7125384	' '17129288	'
'125467988	' '6961162	' '17012161	'
'2259039	' '9194869	' '300000	'
'18023892	' '128325	' '128515	'
'118225	' '129212	' '129608932	'
'128318	' '127896663	' '118194	'
'128226159	' '6830327	' '10728920	'
'2282524	' '130161824	' '120135022	'
'264152806	' '262484031	' '288458324	'

'292559693	' '123901000	' '125966816	'
'17273389	' '125322	' '2297600	'
'128828	' '128431636	' '126989656	'
'128367894	' '125575236	' '129076159	'
'17060552	' '128519	' '129227313	'
'128385579	' '130124155	' '128521	'
'118226	' '124568565	' '129409001	'
'129230	' '129214	' '129238	'
'129245	' '129260	' '129046	'
'126853007	' '129211	' '129265430	'
'127951762	' '51420873	' '129008732	'
'128820858	' '131044292	' '128838	'
'129231	' '129834825	' '127463	'
'17099335	' '60044817	' '120394076	'
'129221	' '118224	' '128600717	'
'129069717	' '131520543	' '131555071	'
'132135953	' '207485404	' '130154709	'
'129248	' '129032447	' '129267	'
'129560263	' '122554867	' '129232	'
'129573021	' '10456666	' '129435307	'
'60083723	' '132582764	' '912027632	'
'131058145	' '50425871	' '207641905	'
'130978754	' '129443405	' '60629424	'
'60238763	' '132474096	' '129293	'
'60540144	' '129263	' '127394	'
'60752334	' '131206940	' '*130953504	'
'130803069	' '130953504	' '120547278	'
'131381511	' '129244	' '131688542	'
'*132582764	' '100138505290790	' '129865649	'
'130679833	' '*60687027	' '130153294	'
'129243	' '456666	' '*127549271	'
'132135944	' '131934036	' '129573030	'
'128509	' '130896814	' '132035749	'
'*60470925	' '60555239	' '131823833	'
'132277989	' '132023946	' '60475832	'
'100385020	' '60317081	' '130813851	'
'60301421	' '60623255	' '60403414	'
'131432084	' '129274	' '9313509	'
'131866683	' '60687027	' '60355829	'
'127218	' '*129766502	' '61002912	'
'17246303	' '60540176	' '130312341	'
'132844765	' '9388090	' '9313510	'
'61076923	' '132967734	' '132973942	'
'131825509	' '*60493022	' '131849954	'
'*132549026	' '*2297600	' '*126853007	'
'129305	' '132245342	' '*60475832	'
'129247	' '129359	' '60465353	'
'*128920633	' '129347	' '134488776	'

'129249047	' '131685389	' '129349	'
'*130577186	' '17101650	' '*131570267	'
'129361	' '132126760	' '133095623	'
'60817488	' '132126470	' '60474003	'
'129766502	' '60077538	' '129433	'
'129430	' '132449988	' '131671771	'
'*132449988	' '*133899281	' '131863578	'
'129076681	' '*61225333	' '60978283	'
'60259622	' '61166977	' '61331399	'
'134120265	' '135460795	' '130198475	'
'61704847	' '51765464	' '101603050	'
'129360	' '7624321769	' '7614571673	'
'64631968	' '122985548	' '130577186	'
'63811814	' '138550394	' '18021429	'
'256096	' '255747	' '60807840	'
'135755633	' '128505	' '128328	'
'6125091528	' '132723	' '135556252	'
'65258473	' '62065919	' '129434	'
'*60807840	' '*60267745	' '137187496	'
'8932162	' '129411	' '*132395373	'
'64775396	' '64541701	' '139383375	'
'65801567	' '64519057	' '65801539	'
'64795909	' '55642031	' '65711256	'
'7891913	' '60372887	' '66591027	'
'*132505846	' '53557278	' '129445	'
'*61174187	' '64519050	' '64700342	'
'52323649	' '62297900	' '60845420	'
'132738781	' '132728	' '137354744	'
'132726	' '136665906	' '207848384	'
'66011448	' '61539377	' '65472563	'
'137477179	' '8932254	' '104854295	'
'136162083	' '138372105	' '207831504	'
'*60641111	' '1001389041906150'	' '135691067	'
'62563777	' '62297890	' '135538586	'
'134774688	' '132749	' '136492474	'
'7611374	' '129412	' '60874781	'
'65621537	' '61547540	' '60597003	'
'129436	' '139325955	' '62294286	'
'129441	' '64633891	' '53573187	'
'105585416	' '50747059	' '132725	'
'135571701	' '129363	' '130588960	'
'62916235	' '61488652	' '61999789	'
'62175933	' '104854291	' '61429186	'
'133126053	' '7927543	' '68718367	'
'65801527	' '132465661	' '132557357	'
'132730	' '52363870	' '135631252	'
'137367221	' '138974820	' '137398410	'
'60996956	' '62555627	' '207641240	'

'64633889	'70098604	'66398553	'
'65689792	'70895035	'136278496	'
'132967761	'64167442	'64541632	'
'62398865	'135872035	'61089329	'
'66591071	'137974489	'132537018	'
'139919013	'133237247	'62398872	'
'62510183	'2211467356	'69474041	'
'139986224	'136770859	'63285238	'
'2238143044	'65801585	'62297888	'
'133759547	'61980169	'7359017	'
'132572570	'131857708	'139005002	'
'65283043	'63031811	'62136638	'
'140199918	'19990190478	'64410464	'
'135747713	'66618935	'66726428	'
'65685163	'61546915	'61898423	'
'50750448	'61802729	'30000346619	'
'5344376730	'62207139	'132777	'
'133299287	'66602381	'62617936	'
'129350	'64990421	'9934676	'
'140206939	'7359243	'138929709	'
'5358071490	'136830917	'522336869	'
'24990074949	'62980834	'7817774	'
'3006777989	'137267222	'65917483	'
'66424345	'66879809	'66019466	'
'137611441	'68886464	'64943912	'
'139575220	'66762643	'68656274	'
'65780183	'3004418246	'66482095	'
'63238367	'139805376	'62661946	'
'63285272	'*132738781	'69056729	'
'136937463	'62784080	'62617973	'
'63262876	'69981346	'105954843	'
'69258330	'62997222	'68771323	'
'138821403	'138027041	'66636663	'
'138425951	'68757314	'10846977	'
'69701759	'62072895	'139456657	'
'69710400	'69957332	'53766368	'
'61898213	'62916225	'62071128	'
'137954458	'138481324	'3009554406	'
'69093267	'3009554392	'69314848	'
'53774726	'3006988835	'3009554395	'
'56496035	'*66011448	'*62563777	'
'11005970	'66406383	'129439	'
'*64633889	'68886414	'3008897582	'
'141092513	'3008897818	'3009554399	'
'3009554407	'68977120	'10918111	'
'140206911	'63555722	'3008328777	'
'*63811814	'3006989495	'3009554388	'
'69701703	'3008897826	'3006972816	'

'65888855	' '*68886414	' '65801591	'
'*62916245	' '69912672	' '70541617	'
'64556559	' '138057109	' '62916245	'
'62030793	' '2251221692	' '63026939	'
'141172974	' '1990208815	' '70377846	'
'66757463	' '69981350	' '3007776352	'
'61488588	' '3008160969	' '64410860	'
'139956906	' '71433746	' '62175749	'
'3007411983	' '65236564	' '69990052	'
'69564829	' '138877011	' '69981349	'
'70541479	' '3000540481	' '*69791520	'
'*70721094	' '129051	' '69708745	'
'11161769	' '69912660	' '71046276	'
'140116973	' '127211	' '128470	'
'129215	' '125167691	' '127778242	'
'108969	' '114866	' '127444	'
'129246	' '121177100	' '2295302	'
'127464	' '125573	' '125677	'
'254209	' '122595334	' '127923558	'
'128664003	' '176138405040444	' '810503	'
'253991	' '10751490	' ']	

AI

[163.409836 164.131147 166.754098 ... 20.590163 21.278688 22.62295]

DI

['forth' 'tirth' 'first' 'second']

MI

[3 4 7 9 10 2 8 6 5 11 12 1]

YI

[1425 1426 1427 1428 1429 1430 1431 1432 1433 1434 1435 1436 1437 1438]

LP

[10 11 7 9 8 12 13 14 6 5 4 3 2 1]

TP

[2 3 1]

S

[120173998 124717307 17256591 126365823 9043859 17058140 17273387
 126941409 125521956 121072478 2290597 2245673 204342911 2280875 17365628
 124109429 6797081 214694884 6754099 125312 127431 17373198 160325582
 120135022 129265430 2234744 127407 300000 125860398 6483076 127390
 123901000 125322 17366195 140459582 288458324 129211 129267 131058145
 60540144 129032447 9313510 129359 125997863 128332 128835 233821937
 128520 120754720 128504 17231270 2301129 128828 128521 2275578 129834825
 129232 129046 128838 127394 129274 128510 128502 129212 128325 128318
 128001 129409001 51420873 118224 17342837 129214 129244 2259250 17012161
 207485404 129260 129263 128509 129238 127442 7125384 127896663 17349617
 125814 129008732 17099335 129221 129243 129347 129349 129430 131555071
 129293 118194 129305 129443405 123982137 125467988 9194869 118225 125674
 17151586 17273389 129230 128515 118226 129245 129231 127463 456666
 159659261 128468 300002 2272874 264152806 129247 131044292 130161824
 121049625 131206940 6780958 6682654 10542523 122185573 2297600 131849954
 132844765 2282524 128385579 126853007 128330 129049 133095623 6975632
 17089950 125178587 292559693 129560263 125575236 2193272 128431636
 130953504 129573021 127218 2157256 2259039 6830327 129435307 60752334
 130803069 123645630 17103268 128519 126989656 120394076 132135953 128501
 130154709 130978754 127951762 124690866 131866683 129076159 129069717
 120614769 2269075 124568565 130124155 10728920 125997890 17302940
 120547278 60317081 60474003 132245342 132023946 129361 129608932 129360
 51765464 129227313 61002912 131823833 122554867 207641905 50425871
 9313509 129249047 130896814 132582764 134488776 129248 60597003 132557357
 61546915 104854295 65258473 52323649 128600717 17101650 132738781
 64700342 129363 60044817 131520543 131825509 132474096 129433 132723
 132726 129411 262484031 17246303 129119896 129766502 64519050 64795909
 132749 132449988 60372887 137267222 128820858 17129288 131184495 60238763
 60403414 137354744 61429186 18023892 60629424 138550394 18021429 62563777
 136162083 130577186 128226159 60355829 128328 61999789 128335 130153294
 129445 132728 3006777989 132777 3004418246 132126470 134774688 60083723
 60540176 128505 134120265 129434 132135944 61166977 255747 131863578
 6125091528 60807840 137477179 50747059 69981346 69701759 17013604
 137187496 60555239 135460795 130198475 128368028 131432084 61704847
 131671771 1001385052907900 128514 131934036 64775396 132973942 64541701
 139805376 8932254 6831072 60687027 66019466 62617973 61547540 69701703
 256096 130679833 132126760 66636663 65472563 17356591 129076681 61539377
 68656274 65780183 2290596 132035749 60978283 135691067 66011448 53557278
 62071128 71046276 60817488 9388090 53766368 139456657 62916225 3009554399
 130813851 135872035 6961162 68757314 62072895 138821403 65975296
 107732708 'S47682286 ' 'S47673325 ' 136937463 61331399
 137974489 105954843 132537018 61898213 136665906 62065919 140206939
 63238367 10846977 61076923 138372105 125966816 56496035 3009554392
 68886414 8932162 128367894 62980834 53774726 3009554406 60623255 63262876
 69957332 53573187 60845420 60077538 69314848 64631968 103102 128332587
 132967734 141172974 70377846 69056729 3009554395 60465353 135747713
 66602381 100 62661946 133126053 60259622 62916235 69258330 69981349
 11491684 69710400 66406383 69096070 65685163 100385020 140206911

107546878 3006988835 70512307 3008609210 11005970 66762643 69912660
 3009553029 139919013 63026939 3013841879 56650817 70640251 60301421
 140969366 3013841912 11595003 3125982660 69990052 3008897699 62784080
 65801585 65888855 64410860 3008897582 3006989495 70626148 71088761
 68767057 60475832 69981350 141092513 129439 3008897826 137954458
 3008897818 138481324 '*69791520 ' 3011816338 207848384 70476856
 66757463 66424345 70953750 3013841922 3124584810 64556559 104936146
 139575220 62669950 69912642 3013001380 69093267 3000540481 3013474523
 3009554407 62997222 138027041 3008167883 69169673 138425951
 'S68533322 ' 522336869 71764603 73963355 60447479 69529197 65711256
 62294286 130312341 71181786 1451884517 3013841921 70726926 3124584862
 'M7402472623 ' 69791579 3013001412 3008328777 71637336 140199918
 71181726 3013001478 70346661 137611441 70801850 71813421 72754110
 69763386 68977120 765206109 69908769 65974975 71129534 3008710387
 3013841929 5368980695 3011733777 70625807 69092963 71922078 3010660326
 70625790 70672880 '*70721094 ' 3009524055 71088613 888659628
 70432608 3013841900 70541498 71974231 'A106935 ' 3013841935
 71703339 3011538150 108700965 71922072 101603050 71753166 72053370
 'A304251 ' 69560690 70626131 63811814 '*72128196 '
 's47673325 ' 140116973 3124720432 137611447 3013071694 11161769
 '*7072194 ' 68656224 71753192 68886532 71922147 63555722
 5363617632 '*72128216 ' 72190820 72451544 3126539606
 '*72128234 ' 'A562106 ' 3125066315 3129016370 3129037786]

AIT

[1011 1010 -1 1017 1027 1026 1012 1028 1030 1018 1031 1015 1034 1033 1013
 1029 '.' 1038 1025 1045 1043 1040 1044 1046 1042 1021 1048 1054 1014 1049
 1035 1019 1052 1050 1051 1032 1047 1053]

DM

[339 361 47 85 102 129 74 79 178 240 91 205 263 67 80 128 151 176
 42 267 172 224 253 274 44 121 166 197 235 257 277 299 334 89 75 149
 187 241 303 380 403 99 124 145 193 216 106 212 114 148 300 379 78 76
 118 122 155 200 242 284 310 116 137 181 202 256 50 28 66 90 123 95
 125 204 111 133 53 101 140 51 71 131 158 182 183 225 248 269 104 135
 157 203 77 234 270 291 45 83 154 201 223 196 243 280 302 69 134 230
 55 156 246 265 286 307 144 232 165 194 110 57 168 93 82 160 189 233
 271 112 150 179 49 281 97 73 65 173 195 217 238 152 210 161 142 226
 273 308 477 497 538 215 388 409 169 323 373 254 366 391 68 175 229 250
 294 316 352 375 411 437 480 132 191 272 327 351 115 177 260 283 52 92
 120 252 138 164 292 317 341 365 438 462 498 146 171 228 289 315 372 105
 214 247 43 184 268 288 72 94 107 219 109 103 61 46 208 293 314 143
 163 81 139 188 394 417 64 333 98 259 130 275 96 295 192 87 369 170
 220 251 56 63 88 119 364 381 127 304 298 186 239 70 261 174 199 321
 86 190 213 117 100 207 231 222 353 54 262 330 206 249 305 328 374 167
 406 147 41 84 180 209 313 377 461 488 511 534 574 596 620 266 337 287

```

48 108 162 60 624 650 464 521 159 211 227 126 136 38 141 59 39 113
320 340 185 236 218 384 245 237 198 342 436 306 296 40 343 255 264 285
344 345 290 153 324 332 62 350 325 371 420 440 484 507 279 244 297 389
412 312 336 382 348 395 429 452 481 276 309 58 221 37 331 338 326 418
439 398 423 402 278 329 408 431 346 349 487 34 367 347 405 425 258 335
370 393 419 442 478 282 448 387 502 524 390 430 386 410 479 36 444 311
424 455 449 485 376 318 32 358 446 564 354 359 523 426 401 378 414 355
363 422 482 508 561 582 609 319 472 356 360 404 441 527 545 629 645 667
687 30 396 357 322 362 33 459 492 301 435 415 392 576 659 679 400 31
443 447 483 525 569 428 453 35 499 526 432 467 397 454 476 29 533 470
383 469 490 475 528 539 567 588 633 427 450 474 445 456 468 493 570 617
416 558 578 605 495 690 724 517 385 460 542 421 473 501 694 529 368 433
457 399 463 509 544 407 504 465 491 551 503 466 550 577 598 612 672 486
543 535 513 602 623 451 458 505 611 434 562 512 496 584 631 573 553 514
518 413 621 547 532 595 643 665 806 565 563 648 673 585 19 489 581 604
670 516 735 764 784 816 530 559 549 572 566 674 506 519 24 589 494 546
568 610 600 586 26 587 619 634 557 641 676 722 607 608 579 632 661 656
510 531 575 597 537 616 683 515 556 580 654 718 767 540 560 552 810 522
684 715 471 696 593 675 793 536 520 555 689 27 671 583 591 640 664 571
635 548 599 614 662 590 707 630 639 755 756 655 741 771 790 638 636 814
646 669 758 783 603 622 666 500 627 601 711 751 681 699 723 625 554 615
613 14 658 618 23 688 734 729 541 726 25 13]

```

AM

```
[19.6 17.2 27.4 ... 54.445 56.43 64.31]
```

CL

```
[21 2 1 11]
```

CI

```
[2 4 3 1]
```

PS

```
[0 1 2]
```

```
[ ]: dataset_copy["SEX"].value_counts()#check the number of samples in each category
      ↪of the SEX column and name of categories
```

```
[ ]: SEX
m      60110
f      54947
```

```

2      1936
2f      756
2m      724
9       523
Name: count, dtype: int64

```

```
[ ]: dataset_copy["DI"].value_counts()#check the number of samples in each category
      ↪of the DI column and name of categories
```

```
[ ]: DI
      tirth      32495
      forth      30478
      first      28058
      second     27965
Name: count, dtype: int64

```

```
[ ]: def categorical_to_numerical(df):
      """
      This function maps non_numeric categories in
      the SEX and DI columns to theirs corresponding
      numeric values
      df:dataset
      """

      column1 = "SEX"
      column2 = "DI"
      label_map1 = {"m":0,"f":1,"2":2,"2m":3,"2f":4}
      label_map2 = {"first":0,"second":1,"tirth":2,"forth":3}
      df[column1] = df[column1].map(label_map1)
      df[column2] = df[column2].map(label_map2)

      return df

```

```
dataset_copy = categorical_to_numerical(dataset_copy)
```

```
[ ]: columns_to_convert = ['SEX', 'WB', 'DS', 'RP', 'IFI', 'DIM', 'SIR',
                           'DI', 'S', 'AIT', 'AM']

for col in columns_to_convert: #convert other non_numeric columns data to
      ↪numeric data
    dataset_copy[col] = pd.to_numeric(dataset_copy[col], errors='coerce')
```

```
[ ]: dataset_copy.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 118996 entries, 0 to 118995
Data columns (total 29 columns):

```

#	Column	Non-Null Count	Dtype
0	Animal	118996 non-null	int64
1	PAS	118996 non-null	int64
2	BT	118996 non-null	int64
3	SEX	118473 non-null	float64
4	WB	117069 non-null	float64
5	DS	118986 non-null	float64
6	RP	117087 non-null	float64
7	TNR	118996 non-null	int64
8	DR	118996 non-null	int64
9	GL	118996 non-null	int64
10	DMP	118996 non-null	int64
11	DFS	118996 non-null	int64
12	IFI	3881 non-null	float64
13	NPA	118996 non-null	int64
14	DIM	115445 non-null	float64
15	SIR	115005 non-null	float64
16	AI	118996 non-null	float64
17	DI	118996 non-null	int64
18	MI	118996 non-null	int64
19	YI	118996 non-null	int64
20	LP	118996 non-null	int64
21	TP	118996 non-null	int64
22	S	117088 non-null	float64
23	AIT	118989 non-null	float64
24	DM	118996 non-null	int64
25	AM	118768 non-null	float64
26	CL	118996 non-null	int64
27	CI	118996 non-null	int64
28	PS	118996 non-null	int64

dtypes: float64(11), int64(18)

memory usage: 26.3 MB

```
[ ]: dataset_copy.isnull().sum()#counts number of null data for each column
```

```
[ ]: Animal      0
      PAS        0
      BT         0
      SEX       523
      WB       1927
      DS        10
      RP       1909
      TNR        0
      DR         0
      GL         0
      DMP        0
```

```

DFS          0
IFI         115115
NPA          0
DIM         3551
SIR         3991
AI           0
DI           0
MI           0
YI           0
LP           0
TP           0
S           1908
AIT          7
DM           0
AM          228
CL           0
CI           0
PS           0
dtype: int64

```

```

[ ]: columns_to_check = ['WB', 'RP', 'AM', 'SEX', 'DS', "IFI"]
null_percentages = {}
for col in columns_to_check: # prints the percentage of null data for columns
    ↪ in columns_to_check
    null_count = dataset_copy[col].isnull().sum()
    total_count = len(dataset_copy[col])
    percent = (null_count / total_count) * 100
    null_percentages[col] = percent

```

```

for col, percent in null_percentages.items():
    print(f" the col {col} has {percent:2f}% null data")

```

```

the col WB has 1.619382% null data
the col RP has 1.604256% null data
the col AM has 0.191603% null data
the col SEX has 0.439511% null data
the col DS has 0.008404% null data
the col IFI has 96.738546% null data

```

```

[ ]: dataset_copy = dataset_copy.drop(columns=["Animal", "IFI", "DIM", "SIR", "AIT",
    ↪ "S"])#removes columns that contains coded or identifier data

```

```

[ ]: dataset_copy = dataset_copy.dropna(subset=['SEX', 'DS', 'WB', 'RP',
    ↪ 'AM'])#removes null values from the specified columns

```

```

[ ]: dataset_copy.isnull().sum()# counts the number of null values for each column

```

```
[ ]: PAS    0
      BT     0
      SEX    0
      WB     0
      DS     0
      RP     0
      TNR    0
      DR     0
      GL     0
      DMP    0
      DFS    0
      NPA    0
      AI     0
      DI     0
      MI     0
      YI     0
      LP     0
      TP     0
      DM     0
      AM     0
      CL     0
      CI     0
      PS     0
      dtype: int64
```

```
[ ]: dataset_copy["PS"].value_counts()#counts the number of values for each category
      ↪in target column
```

```
[ ]: PS
      0    80510
      1    34508
      2         1
      Name: count, dtype: int64
```

```
[ ]: index_to_drop = dataset_copy.index[dataset_copy["PS"] == 2] # removes incorrect
      ↪category from target column
      dataset_copy.drop(index=index_to_drop, inplace=True)
```

```
[ ]: dataset_copy.duplicated().any() #examines the dataset if there is any
      ↪duplicated data
```

```
[ ]: np.True_
```

```
[ ]: dataset_copy = dataset_copy.drop_duplicates()#removes duplicated data from
      ↪dataset
```

```
[ ]: dataset_copy.describe()#provides summary statistics of the dataset's features
```

```
[ ]:
```

	PAS	BT	SEX	WB	DS
RP	TNR	DR	GL	DMP	DFS
NPA	AI	DI	MI	YI	LP
TP	DM	AM	CL	CI	PS
count	115004.000000	115004.000000	115004.000000	115004.000000	115004.000000
115004.000000	115004.000000	115004.000000	115004.000000	115004.000000	
115004.000000	115004.000000	115004.000000	115004.000000	115004.000000	
115004.000000	115004.000000	115004.000000	115004.000000	115004.000000	
115004.000000	115004.000000	115004.000000			
mean	1.960523	1.024686	0.541407	43.775749	1.236087
0.160299	0.599257	43.876796	276.882213	64.570102	
441.750287	0.124448	50.124532	1.549303	6.462888	
1432.153464	2.534921	2.007765	146.891986	40.578417	
8.356335	2.816406	0.299972			
std	0.194727	0.155671	0.624741	13.478930	0.476293
0.366885	1.144765	42.455473	7.743487	27.449279	
35.381076	0.383536	23.260985	1.109711	3.561883	
3.819209	1.666964	0.088172	98.798850	10.768800	
8.019247	0.937346	0.458247			
min	1.000000	1.000000	0.000000	0.000000	1.000000
0.000000	0.000000	0.000000	29.000000	-43.000000	
76.000000	0.000000	16.688524	0.000000	1.000000	
1425.000000	1.000000	1.000000	13.000000	0.100000	
1.000000	1.000000	0.000000			
25%	2.000000	1.000000	0.000000	40.000000	1.000000
0.000000	0.000000	0.000000	274.000000	47.000000	
421.000000	0.000000	31.311475	1.000000	3.000000	
1429.000000	1.000000	2.000000	71.000000	33.800000	
1.000000	2.000000	0.000000			
50%	2.000000	1.000000	0.000000	43.000000	1.000000
0.000000	0.000000	50.000000	277.000000	57.000000	
436.000000	0.000000	43.770491	2.000000	6.000000	
1432.000000	2.000000	2.000000	118.000000	40.800000	
2.000000	2.000000	0.000000			
75%	2.000000	1.000000	1.000000	47.000000	1.000000
0.000000	1.000000	61.000000	281.000000	72.000000	
461.000000	0.000000	62.065573	3.000000	10.000000	
1436.000000	3.000000	2.000000	196.000000	47.800000	
11.000000	4.000000	1.000000			
max	2.000000	3.000000	4.000000	783.000000	4.000000
1.000000	10.000000	632.000000	333.000000	671.000000	
1153.000000	5.000000	183.344262	3.000000	12.000000	
1438.000000	14.000000	3.000000	814.000000	130.100000	
21.000000	4.000000	1.000000			

```
[ ]: negative_values = dataset_copy[dataset_copy['DMP'] < 0] # finds the number of
      ↪negative values in the column DMP
```

```
num_negative = negative_values.shape[0]
print(f"number of nagative values in column DMP is {num_negative}")
```

number of nagative values in column DMP is 10

```
[ ]: negative_index = dataset_copy.index[dataset_copy["DMP"] < 0] #removs the
      ↪negative values from the column DMP
dataset_copy.drop(index=negative_index, inplace=True)
```

```
[ ]: for column in dataset_copy.columns:
      print(f"{column}")
      print(dataset_copy[column].unique())
      print("\n")
```

PAS

[2 1]

BT

[1 2 3]

SEX

[1. 0. 3. 4. 2.]

WB

```
[ 41.    40.    45.    46.    44.    43.    29.    38.    35.    48.
  47.    50.    49.    52.    60.    51.    42.    39.    37.    53.
  55.    77.    63.    74.    62.    58.    34.    33.    81.    36.
  57.    30.    66.    61.    67.    76.    56.    28.     4.    26.
  32.    18.    78.    31.    23.    88.    73.    54.    59.    27.
  13.     3.6    64.    21.    84.   783.    22.    70.    12.    69.
  75.    82.    38.8   72.    80.    79.    24.     4.4    68.    71.
  90.    65.    85.    20.    91.    25.    11.     0.    38.1  273.
249.   307.   234.    87.     3.7   19.    83.     3.2  286.     2.7
  15.    93.     2.9  413.    86.    92.    13.4   38.9    17.    38.2
 39.5   40.2     2.   630.   38.6  637.   121.   145.   138.   115.
111.    74.3  120.     7.     3.1   44.5  122.     1.47  160.    10.
135.   145.5  142.   365.     3.4   98.5  118.     1.42  47.75  10.2
 38.7    14.   662.  160.4  454.   139.   39.2  358.    38.3  131.5
 39.1  144.5    1.45  114.     1.4  671.   455.   96.1     1.  129.5
485.   112.   575.  155.4   51.5   38.5  174.4   89.   168.   110.
256.    82.75  44.4  108.    43.5  138.6  126.8   16.   137.  131.6
126.6  483.   115.6   47.5  207.   163.4   40.7  148.   127.6   82.4
125.3     3.  ]
```


DS

[2. 1. 3. 4.]

RP

[1. 0.]

TNR

[9 8 7 10 4 6 5 2 1 3 0]

DR

[54 111 85 57 119 115 79 132 60 182 36 87 72 167 64 62 70 68
56 61 63 52 67 58 69 65 66 73 100 230 55 160 128 77 98 75
137 105 102 71 241 46 50 141 168 135 121 122 90 106 59 210 109 206
92 237 157 95 81 347 53 124 104 202 153 172 180 107 197 88 261 78
162 125 127 149 265 34 146 49 143 101 118 37 89 221 41 243 152 136
9 76 193 51 96 97 48 133 173 134 83 47 144 234 99 19 91 223
165 148 250 219 246 181 30 74 82 86 142 145 273 138 155 15 114 228
211 84 245 154 94 24 33 112 189 103 208 203 40 28 43 271 130 176
150 44 126 123 45 38 187 171 80 93 175 31 215 116 129 16 260 29
117 201 151 140 244 205 249 170 35 179 8 169 213 218 39 113 42 192
204 226 159 267 110 195 161 164 131 183 214 252 216 199 185 282 313 262
32 174 158 235 177 108 0 259 251 212 184 25 120 321 1 147 186 224
20 163 139 26 156 18 23 21 188 14 236 198 229 217 286 318 11 27
253 10 232 354 207 178 389 288 255 17 190 191 263 375 196 5 293 248
22 225 7 289 247 194 13 350 575 281 239 166 426 3 220 256 266 6
242 12 227 238 2 200 632 222 233 275 4 359 209 307 240 336 504]

GL

[273 281 282 275 278 283 279 280 290 255 276 277 285 287 286 272 270 268
274 288 265 284 289 266 271 264 259 291 267 269 229 29 252 292 230 253
293 256 294 258 261 295 247 209 262 263 297 260 312 243 244 241 254 225
257 232 234 233 250 299 298 227 245 246 251 226 248 238 296 304 249 237
308 236 242 88 143 153 240 231 202 224 320 309 235 210 239 221 129 310
300 218 223 216 212 301 205 307 303 220 213 222 311 305 194 104 228 219
172 302 151 149 333 306 160 154]

DMP

[94 74 79 46 77 91 67 58 42 63 172 44 65 99 114 60 113 83
78 57 50 28 75 51 53 101 200 71 45 69 55 144 66 110 93 82
49 111 54 90 73 102 76 52 106 87 43 183 40 103 81 80 64 48
134 61 56 47 119 166 123 127 70 105 118 88 72 41 130 100 89 39
92 107 38 59 139 236 95 68 136 104 117 108 151 84 140 124 62 86
126 85 37 162 120 129 34 128 96 131 36 191 112 116 182 97 192 32]

```

115 121 132 158 185 472 141 275 33 98 109 31 226 153 163 133 35 148
212 150 30 135 29 161 169 173 186 152 175 285 155 171 149 159 125 165
176 413 328 122 245 376 216 206 235 142 179 189 219 157 174 208 147 210
234 230 156 190 160 138 187 188 221 137 227 170 145 253 164 461 194 146
24 246 143 207 316 26 180 184 178 220 224 202 181 167 197 306 217 293
209 349 205 27 671 222 196 249 154 321 498 198 273 168 256 259 218 295
201 215 242 225 276 14 258 281 195 232 298 290 213 269 228 177 266 338
251 303 203 243 250 199 231 370 23 241 193 291 233 369 238 260 244 264
482 354 240 25 336 13]

```

DFS

```

[ 449 473 485 479 499 510 482 475 501 495 498 532 529 497
504 487 491 500 493 486 490 492 494 513 496 506 508 488
511 489 503 519 502 507 700 552 524 471 535 452 509 1120
1153 457 542 520 610 512 538 523 531 518 517 514 527 423
553 391 534 521 526 515 516 541 543 528 525 546 560 522
533 536 530 602 609 612 603 572 569 554 599 565 588 555
505 623 559 583 557 564 540 585 558 549 551 562 459 548
580 537 484 341 561 556 589 462 291 576 481 469 476 467
478 461 456 454 474 460 464 458 470 466 455 465 463 453
468 293 448 472 483 447 444 439 442 450 433 446 445 451
432 429 436 440 426 420 441 430 443 431 435 438 437 428
427 434 608 422 409 402 397 395 393 425 480 477 278 421
416 412 414 655 539 410 581 545 396 403 367 424 593 665
573 574 571 566 605 384 639 417 418 419 415 563 669 763
627 413 670 570 392 577 405 411 407 408 406 343 386 586
747 575 201 200 221 404 332 401 400 390 398 383 890 807
889 803 844 787 843 786 399 394 370 388 579 594 568 550
366 544 376 387 378 78 375 380 347 374 377 389 381 354
385 382 379 371 76 373 372 369 365 368 364 344 254 633]

```

NPA

```
[0 1 2 3 4 5]
```

AI

```
[163.409836 164.131147 159.57377 ... 20.590163 21.278688 22.62295 ]
```

DI

```
[3 2 0 1]
```

MI

```
[ 3 4 9 2 8 10 6 5 7 11 12 1]
```

YI

[1425 1426 1427 1428 1429 1430 1431 1432 1433 1434 1435 1436 1437 1438]

LP

[10 11 7 9 8 12 13 14 6 5 4 3 2 1]

TP

[2 3 1]

DM

[339 361 74 79 178 240 91 205 263 67 80 42 267 172 224 253 274 44
121 166 197 235 257 277 299 334 89 149 187 241 303 380 403 99 124 145
193 216 114 148 300 379 78 76 118 116 137 181 202 256 50 28 66 90
123 155 75 95 111 133 53 101 140 51 200 71 106 131 158 182 212 183
204 225 248 269 104 135 157 203 77 234 270 291 242 45 83 154 201 223
196 243 280 302 69 134 230 55 156 246 265 286 307 144 232 165 194 110
57 168 93 82 160 189 233 271 112 150 179 49 151 176 281 97 73 65
152 210 129 161 142 226 273 308 477 497 538 215 388 409 102 169 323 373
254 366 391 68 175 229 250 294 316 352 375 411 437 480 132 173 191 272
327 351 115 177 260 283 52 92 120 252 138 164 292 317 341 365 438 462
498 146 171 228 289 315 372 105 214 247 43 184 268 288 72 94 107 128
125 219 109 103 61 46 208 293 314 85 143 163 81 139 188 394 417 64
333 98 259 284 130 275 96 295 192 87 56 63 88 119 47 364 381 127
298 186 239 70 122 261 174 199 321 86 190 213 117 100 207 231 222 353
206 249 305 328 374 167 220 406 147 41 84 209 313 377 461 488 511 534
574 596 620 266 337 287 48 108 162 54 60 624 650 464 521 159 211 227
126 136 180 38 141 59 39 113 170 185 236 195 218 384 245 237 304 198
342 436 262 306 296 40 343 251 255 264 285 310 344 345 290 153 324 332
62 350 279 244 297 389 412 312 336 382 325 348 395 429 452 481 276 309
58 221 37 331 338 326 369 418 439 398 423 217 402 278 329 408 431 346
320 349 487 34 367 258 335 370 393 419 442 478 282 448 387 502 524 390
386 410 238 405 479 347 36 311 340 424 455 376 318 32 358 446 564 354
359 523 426 330 401 378 414 355 363 422 482 508 561 582 609 319 472 356
360 404 441 527 545 629 645 667 687 396 357 322 362 33 420 459 485 492
301 371 435 415 392 576 659 679 400 31 443 447 483 525 569 428 453 35
430 425 499 526 432 467 30 397 454 476 29 533 470 383 469 490 475 528
539 567 588 633 427 450 474 445 456 468 493 570 617 416 440 558 578 605
495 690 724 444 517 385 421 473 501 694 529 368 484 433 457 399 463 509
460 544 407 504 465 491 551 466 550 577 598 612 672 486 543 535 513 602
623 507 451 449 458 505 611 434 562 512 496 584 631 503 573 514 413 621
547 532 595 643 665 806 565 585 489 581 604 648 518 670 516 530 549 572
566 674 506 519 24 589 494 542 600 586 26 587 619 634 557 641 676 722
607 608 579 632 661 656 510 531 575 597 515 556 559 580 654 718 767 540
560 552 810 522 684 715 471 696 675 793 536 520 537 555 689 27 671 583]

```

591 640 664 571 563 635 548 599 614 590 630 568 755 756 593 655 741 771
790 638 636 546 814 646 669 758 783 610 603 622 666 500 627 601 711 751
681 625 554 615 639 613 14 658 618 23 662 688 734 729 541 726 25 13]

```

```

AM
[19.6 17.2 30.4 ... 54.445 56.43 64.31 ]

```

```

CL
[21 2 1 11]

```

```

CI
[2 4 3 1]

```

```

PS
[0 1]

```

```

[ ]: #groups the dataset by the BT column and lists the unique values of the SEX_
      ↪column within each group
dataset_copy.groupby("BT")["SEX"].unique()

```

```

[ ]: BT
1    [1.0, 0.0, 2.0, 3.0]
2    [3.0, 4.0, 2.0]
3    [1.0, 2.0]
Name: SEX, dtype: object

```

```

[ ]: dataset_copy[dataset_copy["BT"]==3] #displays the rows of the dataset where the_
      ↪BT column has a value of 3

```

```

[ ]:
      PAS BT SEX  WB DS RP TNR DR GL DMP DFS NPA AI DI
MI  YI LP TP  DM  AM CL CI PS
52265  2 3 1.0 24.0 1.0 1.0 2 97 256 54 488 0 59.147540 0
10 1432 3 2 54 51.80 11 2 0
52266  2 3 1.0 24.0 1.0 1.0 2 97 256 54 488 0 60.655737 2
11 1432 3 2 100 45.20 21 2 0
52267  2 3 1.0 24.0 1.0 1.0 2 97 256 54 488 0 61.573770 2
12 1432 3 2 128 37.40 21 4 1
52268  2 3 1.0 24.0 1.0 1.0 2 97 256 54 488 1 63.409836 2
2 1433 3 2 184 42.20 2 4 1
100525 2 3 2.0 36.0 1.0 1.0 1 46 270 51 437 0 36.852459 2
11 1436 2 2 51 49.40 21 3 0
100526 2 3 2.0 36.0 1.0 1.0 1 46 270 51 437 0 40.360655 1

```

```

3  1437  2  2  158  52.05  11  3  0
100527  2  3  2.0  36.0  1.0  1.0  1  46  270  51  437  0  42.754098  3
5  1437  2  2  231  27.90  21  4  0
100528  2  3  2.0  36.0  1.0  1.0  1  46  270  51  437  0  44.426229  2
7  1437  2  2  282  44.80  2  2  0
100529  2  3  2.0  36.0  1.0  1.0  1  46  270  51  437  0  45.868852  3
8  1437  2  2  326  35.30  1  2  0

```

```

[ ]: #displays the rows of the dataset where the BT column has a value of 1 and the
      ↪Sex column has a value of 2
dataset_copy[(dataset_copy["BT"]==1) & (dataset_copy["SEX"]==2)]

```

```

[ ]:
      PAS BT SEX  WB DS RP TNR DR GL DMP DFS NPA AI
DI MI  YI LP TP  DM AM CL CI PS
5235  2  1  2.0  41.0  1.0  0.0  4  59  277  61  489  0  85.475409
2  10  1427  6  2  61  43.7  2  2  1
5957  2  1  2.0  40.0  1.0  0.0  4  47  270  46  467  0  105.573770
2  11  1429  7  2  46  49.9  2  2  0
5958  2  1  2.0  40.0  1.0  0.0  4  47  270  46  467  0  106.295081
0  12  1429  7  2  68  57.8  11  2  0
5959  2  1  2.0  40.0  1.0  0.0  4  47  270  46  467  0  109.737704
3  3  1430  7  2  173  55.2  21  2  1
12213  2  1  2.0  42.0  1.0  0.0  3  61  284  40  456  0  98.196721
1  12  1430  7  2  40  18.8  11  2  1
...  ...  ..  ...  ...  ...  ...  ...  ...  ...  ...
..  ..  ...  ..  ..  ...  ..  ..  ..
118319  2  1  2.0  34.0  1.0  0.0  0  52  274  54  473  0  63.704918
2  6  1433  4  2  165  54.0  2  2  1
118355  2  1  2.0  34.0  1.0  0.0  1  110  273  56  467  2  58.393442
2  1  1433  3  2  56  50.2  21  2  0
118356  2  1  2.0  34.0  1.0  0.0  1  110  273  56  467  2  61.245901
2  4  1433  3  2  143  29.2  21  3  0
118357  2  1  2.0  34.0  1.0  0.0  1  110  273  56  467  2  62.655737
0  6  1433  3  2  186  42.6  11  4  0
118358  2  1  2.0  34.0  1.0  0.0  1  110  273  56  467  2  63.639344
0  7  1433  3  2  216  32.2  11  2  0

```

[398 rows x 23 columns]

```

[ ]: COUNT = dataset_copy[(dataset_copy["BT"]==1) & (dataset_copy["SEX"]==2)].
      ↪shape[0]
print(COUNT)#counts the rows of the dataset where the BT column has a value of
      ↪1 and the Sex column has a value of 2

```

398

```
[ ]: COUNT1 = dataset_copy[(dataset_copy["BT"]==1) & (dataset_copy["SEX"]==3)].
      ↪shape[0]
      print(COUNT1)#counts the rows of the dataset where the BT column has a value of 1
      ↪and the Sex column has a value of 3
```

2

```
[ ]: dataset_copy =dataset_copy[dataset_copy["BT"] != 3] #removes the rows of the
      ↪dataset where the BT column has a value pf 3
```

```
[ ]: #removes the rows of the dataset where the BT column has a value of 1 and the
      ↪Sex column has a value of 3
      dataset_copy = dataset_copy[~((dataset_copy["BT"]==1) &
      ↪(dataset_copy["SEX"]==3))]
```

```
[ ]: #removes the rows of the dataset where the BT column has a value of 1 and the
      ↪Sex column has a value of 2
      dataset_copy = dataset_copy[~((dataset_copy["BT"]==1) &
      ↪(dataset_copy["SEX"]==2))]
```

```
[ ]: dataset_copy
```

```
[ ]:
      PAS BT SEX WB DS RP TNR DR GL DMP DFS NPA AI
DI MI YI LP TP DM AM CL CI PS
0 2 1 1.0 41.0 2.0 1.0 9 54 273 94 449 0 163.409836
3 3 1425 10 2 339 19.60 21 2 0
1 2 1 1.0 41.0 2.0 1.0 9 54 273 94 449 0 164.131147
2 4 1425 10 2 361 17.20 2 2 0
6 2 1 0.0 40.0 2.0 1.0 8 111 281 74 473 1 159.573770
2 3 1425 10 2 74 30.40 21 2 0
7 2 1 0.0 40.0 2.0 0.0 7 85 282 79 473 0 157.475409
0 9 1425 11 2 79 36.10 1 2 1
8 2 1 0.0 41.0 2.0 1.0 10 57 275 46 485 0 147.442622
0 2 1425 10 2 178 30.60 1 2 0
... ..
.. ..
118990 2 1 1.0 44.0 2.0 0.0 0 95 279 71 445 0 85.049180
3 4 1436 5 2 224 24.80 1 2 0
118991 2 1 1.0 44.0 2.0 0.0 0 95 279 71 445 0 85.836065
2 5 1436 5 2 248 23.25 21 4 0
118992 2 1 1.0 44.0 2.0 0.0 0 95 279 71 445 0 87.409836
1 7 1436 5 2 296 25.10 11 2 0
118993 2 1 0.0 48.0 1.0 0.0 0 0 279 64 437 0 28.163934
0 6 1431 1 2 64 34.80 11 4 1
118994 2 1 1.0 40.0 1.0 0.0 0 58 280 49 437 0 38.950819
1 5 1432 2 2 49 59.90 11 2 1
```

[114585 rows x 23 columns]

```
[ ]: dataset_copy.to_excel("/content/drive/MyDrive/project_first/cleared_data.xlsx",
    ↪index=False) # saves cleared data
```

```
[ ]: file_path = "/content/drive/MyDrive/project_first/cleared_data.xlsx"
dataset_cleared = pd.read_excel(file_path) # displays cleared data
pd.set_option("display.max_columns",None) # for showing all columns
pd.set_option("display.expand_frame_repr",False) # to avoid data clutter
dataset_cleared
```

```
[ ]:
```

	PAS	BT	SEX	WB	DS	RP	TNR	DR	GL	DMP	DFS	NPA	AI	DI	
MI	YI	LP	TP	DM	AM	CL	CI	PS							
0		2	1	1	41.0	2	1	9	54	273	94	449	0	163.409836	3
3	1425	10	2	339	19.60	21	2	0							
1		2	1	1	41.0	2	1	9	54	273	94	449	0	164.131147	2
4	1425	10	2	361	17.20	2	2	0							
2		2	1	0	40.0	2	1	8	111	281	74	473	1	159.573770	2
3	1425	10	2	74	30.40	21	2	0							
3		2	1	0	40.0	2	0	7	85	282	79	473	0	157.475409	0
9	1425	11	2	79	36.10	1	2	1							
4		2	1	0	41.0	2	1	10	57	275	46	485	0	147.442622	0
2	1425	10	2	178	30.60	1	2	0							
...	...	..	...	...	..	...	...	...	...	...	...	..			
...	...	..	...	...	..	...	...	...							
114580		2	1	1	44.0	2	0	0	95	279	71	445	0	85.049180	3
4	1436	5	2	224	24.80	1	2	0							
114581		2	1	1	44.0	2	0	0	95	279	71	445	0	85.836065	2
5	1436	5	2	248	23.25	21	4	0							
114582		2	1	1	44.0	2	0	0	95	279	71	445	0	87.409836	1
7	1436	5	2	296	25.10	11	2	0							
114583		2	1	0	48.0	1	0	0	0	279	64	437	0	28.163934	0
6	1431	1	2	64	34.80	11	4	1							
114584		2	1	1	40.0	1	0	0	58	280	49	437	0	38.950819	1
5	1432	2	2	49	59.90	11	2	1							

[114585 rows x 23 columns]

```
[ ]: dataset_cleared_copy = dataset_cleared.copy() # makes a copy from dataset
```

```
[ ]: continuous_columns = ["WB", "DR", "GL", "DMP", "DFS", "AI", "DM", "AM"]
    ↪#contains the continuous columns
categorical_columns = ["PAS", "BT", "SEX", "DS", "RP", "TNR", "NPA", "DI", "MI",
    ↪"YI", "LP", "TP", "CL", "CI", "PS"] # contains
    ↪categorical column
```

```
[ ]: def plot_distributions(df, continuous_cols, categorical_cols,
    ↪save_path):
    """
```

```

This function plots the distribution diagrams
(hisplot for continuous columns and count plot for categorical columns)
for all columns of dataset
"""

n_cols = 4
n_rows = 2
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols*4, n_rows*3))
axes = axes.flatten()
colors_cont = sns.color_palette("tab20", len(continuous_cols))
for i, col in enumerate(continuous_cols):
    sns.histplot(df[col], kde=True, bins=30, color=colors_cont[i], ax=axes[i])
    axes[i].set_title(col, fontsize=12, fontweight="bold")
    axes[i].set_xlabel('')
    axes[i].set_ylabel("Count")
    axes[i].grid(axis='y', linestyle='--', alpha=0.7)

for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
if save_path:
    os.makedirs(save_path, exist_ok=True)
    plt.savefig(os.path.join(save_path, "continuous_cols_distributions.png"),
↳dpi=900)
plt.close()
cat_first8 = categorical_cols[:8]
n_cols = 4
n_rows = 2
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols*4, n_rows*3))
axes = axes.flatten()
colors_cat = sns.color_palette("Set3", len(cat_first8))
for i, col in enumerate(cat_first8):
    order = df[col].value_counts().index
    sns.countplot(data=df, x=col, order=order,
↳palette=[colors_cat[i]]*len(order), ax=axes[i])
    axes[i].set_title(col, fontsize=12, fontweight="bold")
    axes[i].set_xlabel('')
    axes[i].set_ylabel("Frequency")
    axes[i].tick_params(axis="x", rotation=45)
    axes[i].grid(axis='y', linestyle='--', alpha=0.7)

for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
if save_path:
    plt.savefig(os.path.join(save_path, "categorical_fisrt8.png"), dpi=900)
plt.close()
cat_remaining = categorical_cols[8:]

```



```

n_cols = 4
n_rows = 2
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols*4, n_rows*3))
axes = axes.flatten()
colors_cat_1 = sns.color_palette("Set3", len(cat_remaining))
for i, col in enumerate(cat_remaining):
    order_2 = df[col].value_counts().index
    sns.countplot(data=df, x=col, order=order_2,
↪palette=[colors_cat_1[i]]*len(order_2), ax=axes[i])
    axes[i].set_title(col, fontsize=12, fontweight="bold")
    axes[i].set_xlabel('')
    axes[i].set_ylabel("Frequency")
    axes[i].tick_params(axis="x", rotation=45)
    axes[i].grid(axis='y', linestyle='--', alpha=0.7)
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
if save_path:
    plt.savefig(os.path.join(save_path, "categorical_remaining7.png"), dpi=900)
plt.close()

save_path1 = "/content/drive/MyDrive/project_first"
plot_distributions(dataset_cleared_copy, continuous_columns,
↪categorical_columns, save_path1)

```

```

[ ]: def plot_skewness_table(df, continuous_cols, save_path=None):
    """
    This function first calculates the skewness for continuous columns
    then makes a table for them
    """

    skew_values = [skew(df[col], bias=False) for col in continuous_cols]
    skew_table = pd.DataFrame({"Feature": continuous_cols, "Skewness":
↪skew_values})
    skew_table["Abs_skew"] = skew_table["Skewness"].abs()
    skew_table.sort_values(by="Abs_skew", ascending=False, inplace=True)
    skew_table.reset_index(drop=True, inplace=True)
    table_matrix = skew_table[["Feature", "Skewness"]].copy()
    table_matrix.set_index("Feature", inplace=True)
    plt.figure(figsize=(max(8, len(continuous_cols)*0.6), 2))
    cmap = sns.diverging_palette(220, 20, as_cmap=True)
    sns.heatmap(table_matrix.T, annot=True, fmt=".3f", cmap=cmap, cbar=True,
                linewidths=0.5, linecolor="gray", center=0,
                annot_kws={"fontsize":11, "fontweight":"bold", "color":"black"})
    plt.title("Skewness of Continuous Features", fontsize=14, fontweight="bold")
    plt.yticks(rotation=0, fontsize=10)

```

```

plt.xticks(rotation=45, ha="right", fontsize=10)
plt.tight_layout()
if save_path:
    os.makedirs(save_path, exist_ok=True)
plt.savefig(os.path.join(save_path, "continuous_skewness_table.png"),
    ↪dpi=900, bbox_inches="tight")
plt.close()
return skew_table

```

```
plot_skewness_table(dataset_cleared_copy, continuous_columns, save_path1)
```

```
[ ]: Feature    Skewness    Abs_skew
0      WB    24.619073    24.619073
1      GL    -5.041256     5.041256
2      DMP     2.775063     2.775063
3      DFS     2.705960     2.705960
4      DR     1.623063     1.623063
5      DM     1.414756     1.414756
6      AI     1.290745     1.290745
7      AM    -0.130945     0.130945

```

```
[ ]: continuous_columns = ["WB", "DR", "GL", "DMP", "DFS", "AI", "DM", "AM"] # ↪
    ↪counts the number of zero values for specified columns
for col in continuous_columns:
    n_zeros = (dataset_cleared_copy[col] == 0).sum()
    if n_zeros > 0:
        print(f"column {col} has {n_zeros} zero data")
    else:
        print(f"column {col} has no zero data")

```

```

column WB has 14 zero data
column DR has 38321 zero data
column GL has no zero data
column DMP has no zero data
column DFS has no zero data
column AI has no zero data
column DM has no zero data
column AM has no zero data

```

```
[ ]: def plot_new_skewness_table(df, continuous_cols, save_path=None):
    """
    This function first calculates the skewness after addressing the issue
    than plots the updated skewness table
    """

```

```

skew_values = [skew(df[col], bias=False) for col in continuous_cols]
skew_table = pd.DataFrame({"Feature": continuous_cols, "Skewness":  

↳skew_values})
skew_table["Abs_skew"] = skew_table["Skewness"].abs()
skew_table.sort_values(by="Abs_skew", ascending=False, inplace=True)
skew_table.reset_index(drop=True, inplace=True)
table_matrix = skew_table[["Feature", "Skewness"]].copy()
table_matrix.set_index("Feature", inplace=True)
plt.figure(figsize=(max(8, len(continuous_cols)*0.6), 2))
cmap = sns.diverging_palette(220, 20, as_cmap=True)
sns.heatmap(table_matrix.T, annot=True, fmt=".3f", cmap=cmap, cbar=True,  

            linewidths=0.5, linecolor="gray", center=0,  

            annot_kws={"fontsize":11, "fontweight":"bold", "color":"black"})
plt.title("Skewness of Continuous Features", fontsize=14)
plt.yticks(rotation=0)
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
if save_path:
    os.makedirs(save_path, exist_ok=True)
plt.savefig(os.path.join(save_path, "continuous_skewness_table_after.png"),  

↳dpi=900, bbox_inches="tight")
plt.close()
return skew_table

```

```

[ ]: # AM columns is approximately normal
# columns WB and DR have zero data
skew_columns = ["WB", "DMP", "GL", "DFS", "DR", "DM", "AI"]
pt = PowerTransformer(method="yeo-johnson", standardize=False)
lambda_yeo_dict = {}
for col in skew_columns: #addresses the issue of skewness in continuous columns  

↳by using yeo-johnson method
    data = dataset_cleared_copy[[col]].values
    dataset_cleared_copy[col] = pt.fit_transform(data)
    lambda_yeo_dict[col] = float(pt.lambdas_[0])

skew_after = [skew(dataset_cleared_copy[col].values, bias=False) for col in  

↳skew_columns]
report = pd.DataFrame({
    "Column": skew_columns,
    "Transformation": ["Yeo-Johnson"] * len(skew_columns),
    "Lambda": [lambda_yeo_dict[col] for col in skew_columns],
    "skewness_after": skew_after
})
print(report)
report.to_csv("/content/drive/MyDrive/project_first/  

↳skewness_transformation_report.csv", index=False)

```

	Column	Transformation	Lambda	skewness_after
0	WB	Yeo-Johnson	0.125694	0.355934
1	DMP	Yeo-Johnson	-0.849891	0.020304
2	GL	Yeo-Johnson	9.691543	0.389441
3	DFS	Yeo-Johnson	-0.000820	-0.000636
4	DR	Yeo-Johnson	0.323128	-0.350530
5	DM	Yeo-Johnson	-0.105640	0.019432
6	AI	Yeo-Johnson	-0.471360	0.063438

```
[ ]: #plots the distribution diagrams for continuous columns after adressig the
      ↪issue
n_cols = 4
n_rows = 2
continuous_columns_1 = ["WB", "DR", "GL", "DMP", "DFS", "AI", "DM", "AM"]
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols*4, n_rows*3))
axes = axes.flatten()
colors_cont_2 = sns.color_palette("tab20", len(continuous_columns_1))
for i, col in enumerate(continuous_columns_1):
    sns.histplot(dataset_cleared_copy[col], kde=True, bins=30,
      ↪color=colors_cont_2[i], ax=axes[i])
    axes[i].set_title(col, fontsize=12, fontweight="bold")
    axes[i].set_xlabel('')
    axes[i].set_ylabel("Count")
    axes[i].grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/
      ↪continuous_cols_distributions_after.png", dpi=900)
plt.close()
```

```
[ ]: new_path = "/content/drive/MyDrive/project_first"
plot_new_skewness_table(dataset_cleared_copy, continuous_columns_1,
      ↪save_path=new_path)
```

```
[ ]: Feature  Skewness  Abs_skew
0      GL  0.389441  0.389441
1      WB  0.355934  0.355934
2      DR -0.350530  0.350530
3      AM -0.130945  0.130945
4      AI  0.063438  0.063438
5      DMP 0.020304  0.020304
6      DM  0.019432  0.019432
7      DFS -0.000636  0.000636
```

```
[ ]: dataset_cleared_copy
```

```
[ ]:
      PAS  BT  SEX      WB  DS  RP  TNR      DR      GL      DMP
DFS  NPA      AI  DI  MI      YI  LP  TP      DM      AM  CL  CI  PS
0      2  1      1  4.770944  2  1  9  8.202830  4.356831e+22  1.152087
6.093963  0  1.930030  3  3  1425  10  2  4.352277  19.60  21  2  0
1      2  1      1  4.770944  2  1  9  8.202830  4.356831e+22  1.152087
6.093963  0  1.930425  2  4  1425  10  2  4.386036  17.20  2  2  0
2      2  1      0  4.732454  2  1  8  11.121507  5.758387e+22  1.146627
6.145662  1  1.927888  2  3  1425  10  2  3.466952  30.40  21  2  0
3      2  1      0  4.732454  2  0  7  9.958425  5.959364e+22  1.148228
6.145662  0  1.926683  0  9  1425  11  2  3.507714  36.10  1  2  1
4      2  1      0  4.770944  2  1  10  8.398384  4.675000e+22  1.132001
6.170537  0  1.920583  0  2  1425  10  2  3.993677  30.60  1  2  0
...      ... .. ...      ... .. .. ...      ...      ...
... ..      ... .. ..      ... .. ..      ...      ... .. .. ..
114580  2  1      1  4.881791  2  0  0  10.430736  5.374568e+22  1.145569
6.085079  0  1.861694  3  4  1436  5  2  4.124313  24.80  1  2  0
114581  2  1      1  4.881791  2  0  0  10.430736  5.374568e+22  1.145569
6.085079  0  1.862806  2  5  1436  5  2  4.181202  23.25  21  4  0
114582  2  1      1  4.881791  2  0  0  10.430736  5.374568e+22  1.145569
6.085079  0  1.864987  1  7  1436  5  2  4.278707  25.10  11  2  0
114583  2  1      0  5.019941  1  0  0  0.000000  5.374568e+22  1.142748
6.067069  0  1.688830  0  6  1431  1  2  3.375573  34.80  11  4  1
114584  2  1      1  4.732454  1  0  0  8.462045  5.563510e+22  1.134287
6.067069  0  1.748484  1  5  1432  2  2  3.204406  59.90  11  2  1
```

[114585 rows x 23 columns]

```
[ ]: dataset_cleared_copy.describe()
```

```
[ ]:
      PAS      BT      SEX      WB      DS
RP      TNR      DR      GL      DMP      DFS
NPA      AI      DI      MI      YI      LP
TP      DM      AM      CL      CI      PS
count  114585.000000  114585.000000  114585.000000  114585.000000  114585.000000
114585.000000  114585.000000  114585.000000  1.145850e+05  114585.000000
114585.000000  114585.000000  114585.000000  114585.000000  114585.000000
114585.000000  114585.000000  114585.000000  114585.000000  114585.000000
114585.000000  114585.000000  114585.000000
mean      1.960396      1.024619      0.536178      4.845323      1.236610
0.159881      0.598831      5.780065  5.122359e+22      1.139696
6.074822      0.124397      1.768216      1.549522      6.462111
1432.150788      2.532923      2.007767      3.748319      40.572984
8.353798      2.816058      0.300118
std      0.195027      0.154963      0.619748      0.315259      0.476706
0.366498      1.145566      4.315006  1.079256e+22      0.009710
0.076367      0.383508      0.066937      1.109624      3.561875
3.822121      1.666753      0.088186      0.382980      10.768079
```

8.018813	0.937404	0.458311			
min	1.000000	1.000000	0.000000	0.000000	1.000000
0.000000	0.000000	0.000000	2.133982e+13	1.051724	
4.336075	0.000000	1.573830	0.000000	1.000000	
1425.000000	1.000000	1.000000	2.303115	0.100000	
1.000000	1.000000	0.000000			
25%	2.000000	1.000000	0.000000	4.732454	1.000000
0.000000	0.000000	0.000000	4.513401e+22	1.132792	
6.030040	0.000000	1.709237	1.000000	3.000000	
1429.000000	1.000000	2.000000	3.441025	33.800000	
1.000000	2.000000	0.000000			
50%	2.000000	1.000000	0.000000	4.845579	1.000000
0.000000	0.000000	7.930521	5.013852e+22	1.139304	
6.064795	0.000000	1.767861	2.000000	6.000000	
1432.000000	2.000000	2.000000	3.752494	40.800000	
2.000000	2.000000	0.000000			
75%	2.000000	1.000000	1.000000	4.986355	1.000000
0.000000	1.000000	8.648748	5.758387e+22	1.145930	
6.120149	0.000000	1.820634	3.000000	10.000000	
1436.000000	3.000000	2.000000	4.048790	47.750000	
11.000000	4.000000	1.000000			
max	2.000000	2.000000	4.000000	10.430036	4.000000
1.000000	10.000000	21.784516	2.968955e+23	1.171969	
7.030635	5.000000	1.940086	3.000000	12.000000	
1438.000000	14.000000	3.000000	4.803410	130.100000	
21.000000	4.000000	1.000000			

```
[ ]: continuous_columns_1 = ["WB", "DR", "GL", "DMP", "DFS", "AI", "DM", "AM"]
def plot_boxplots(df, continues_col, n_rows=2, n_cols=4):
    """
    This function plots boxplots for continuous columns and
    identifies outliers using the IQR method
    """
    sns.set(style="whitegrid", font_scale=1.2)
    plt.rcParams.update({
        "axes.edgecolor": "0.45",
        "axes.linewidth": 0.9,
        "grid.color": "0.9",
        "grid.linestyle": "--",
        "grid.linewidth": 0.5
    })
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(5*n_cols, 4*n_rows))
    axes = axes.flatten()
    colors = sns.color_palette("tab20", len(continues_col))
    for i, col in enumerate(continues_col[:n_rows*n_cols]):
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
```

```

IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[col][(df[col] < lower_bound) | (df[col] > upper_bound)]
sns.boxplot(x=df[col], ax=axes[i], color=colors[i], fliersize=4,
            linewidth=1.5, boxprops=dict(alpha=0.8, edgecolor="black"),
            whiskerprops=dict(color="black", linewidth=1.2),
            capprops=dict(color="black", linewidth=1.2),
            medianprops=dict(color="yellow", linewidth=1.5))
axes[i].scatter(outliers, [0]*len(outliers), color="red", zorder=10, s=25,
↳alpha=0.8)
axes[i].set_title(col, fontsize=12, fontweight="bold", pad=8)
axes[i].set_xlabel("")
sns.despine(ax=axes[i], left=True)
for j in range(len(continues_col), len(axes)):
    fig.delaxes(axes[j])
fig.suptitle("Boxplots of Continuous Features", fontsize=15,
↳fontweight="bold", y=0.995)
plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.savefig("/content/drive/MyDrive/project_first/BOXPLOT.png", dpi=900,
↳bbox_inches="tight")
plt.close()

```

```
plot_boxplots(dataset_cleared_copy, continuous_columns_1)
```

```
[ ]: dataset_cleared_copy.to_excel("/content/drive/MyDrive/project_first/
↳new_cleared_data.xlsx", index=False) # saves cleared dataset
```

```
[ ]: file_path = "/content/drive/MyDrive/project_first/new_cleared_data.xlsx"
dataset_cleared_new = pd.read_excel(file_path) #displays cleared data
pd.set_option("display.max_columns",None) # for showing all columns
pd.set_option("display.expand_frame_repr",False) # to avoid data clutter
dataset_cleared_new
```

```
[ ]:
```

	PAS	BT	SEX		WB	DS	RP	TNR		DR		GL		DMP
DFS	NPA		AI	DI	MI	YI	LP	TP		DM	AM	CL	CI	PS
0		2	1	1	4.770944	2	1	9	8.202830	4.356831e+22	1.152087			
6.093963		0	1.930030	3	3	1425	10	2	4.352277	19.60	21	2	0	
1		2	1	1	4.770944	2	1	9	8.202830	4.356831e+22	1.152087			
6.093963		0	1.930425	2	4	1425	10	2	4.386036	17.20	2	2	0	
2		2	1	0	4.732454	2	1	8	11.121507	5.758387e+22	1.146627			
6.145662		1	1.927888	2	3	1425	10	2	3.466952	30.40	21	2	0	
3		2	1	0	4.732454	2	0	7	9.958425	5.959364e+22	1.148228			
6.145662		0	1.926683	0	9	1425	11	2	3.507714	36.10	1	2	1	
4		2	1	0	4.770944	2	1	10	8.398384	4.675000e+22	1.132001			

```

6.170537    0 1.920583    0 2 1425 10 2 3.993677 30.60 1 2 0
... ..
... ..
114580    2 1 1 4.881791 2 0 0 10.430736 5.374568e+22 1.145569
6.085079    0 1.861694 3 4 1436 5 2 4.124313 24.80 1 2 0
114581    2 1 1 4.881791 2 0 0 10.430736 5.374568e+22 1.145569
6.085079    0 1.862806 2 5 1436 5 2 4.181202 23.25 21 4 0
114582    2 1 1 4.881791 2 0 0 10.430736 5.374568e+22 1.145569
6.085079    0 1.864987 1 7 1436 5 2 4.278707 25.10 11 2 0
114583    2 1 0 5.019941 1 0 0 0.000000 5.374568e+22 1.142748
6.067069    0 1.688830 0 6 1431 1 2 3.375573 34.80 11 4 1
114584    2 1 1 4.732454 1 0 0 8.462045 5.563510e+22 1.134287
6.067069    0 1.748484 1 5 1432 2 2 3.204406 59.90 11 2 1

```

[114585 rows x 23 columns]

```
[ ]: dataset_cleared_new_copy = dataset_cleared_new.copy() #makes a copy of dataset
```

```
[ ]: def report_outlier_percentage(df, columns=None):
    """
    This function calculates the percentage of outlier
    values in continuous columns by using IQR method
    """
    for col in columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = ((df[col] < lower_bound) | (df[col] > upper_bound))
        percent = outliers.sum() / len(df) * 100
        print(f"{col}: {percent:6.2f}%")

    continuous_columns_2 = ["WB", "DR", "GL", "DMP", "DFS", "AI", "DM", "AM"]
    report_outlier_percentage(dataset_cleared_new_copy, continuous_columns_2)
```

```

WB: 6.63%
DR: 0.00%
GL: 3.15%
DMP: 0.39%
DFS: 1.79%
AI: 0.00%
DM: 0.00%
AM: 1.12%

```



```
[ ]: def remove_outliers(df, columns):
    """
    This function removes the outliers from continuous columns by using IQR
    """
    for col in columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
    return df

continuous_columns_3 = ["WB", "DR", "GL", "DMP", "DFS", "DM", "AM"]
dataset_cleared_new_copy = remove_outliers(dataset_cleared_new_copy,
↪continuous_columns_3)
```

```
[ ]: dataset_cleared_new_copy
```

```
[ ]:
```

	PAS	BT	SEX		WB	DS	RP	TNR		DR		GL		DMP
DFS	NPA		AI	DI	MI	YI	LP	TP		DM	AM	CL	CI	PS
0		2	1	1	4.770944	2	1	9	8.202830	4.356831e+22	1.152087			
6.093963		0	1.930030	3	3	1425	10	2	4.352277	19.60	21	2	0	
1		2	1	1	4.770944	2	1	9	8.202830	4.356831e+22	1.152087			
6.093963		0	1.930425	2	4	1425	10	2	4.386036	17.20	2	2	0	
2		2	1	0	4.732454	2	1	8	11.121507	5.758387e+22	1.146627			
6.145662		1	1.927888	2	3	1425	10	2	3.466952	30.40	21	2	0	
3		2	1	0	4.732454	2	0	7	9.958425	5.959364e+22	1.148228			
6.145662		0	1.926683	0	9	1425	11	2	3.507714	36.10	1	2	1	
4		2	1	0	4.770944	2	1	10	8.398384	4.675000e+22	1.132001			
6.170537		0	1.920583	0	2	1425	10	2	3.993677	30.60	1	2	0	
...	...	..	...	...	..	..	..	...	...	...	...	...	...	...
...	...	...	..	..	...	..	..	...	...	..	..	..	..	..
114580		2	1	1	4.881791	2	0	0	10.430736	5.374568e+22	1.145569			
6.085079		0	1.861694	3	4	1436	5	2	4.124313	24.80	1	2	0	
114581		2	1	1	4.881791	2	0	0	10.430736	5.374568e+22	1.145569			
6.085079		0	1.862806	2	5	1436	5	2	4.181202	23.25	21	4	0	
114582		2	1	1	4.881791	2	0	0	10.430736	5.374568e+22	1.145569			
6.085079		0	1.864987	1	7	1436	5	2	4.278707	25.10	11	2	0	
114583		2	1	0	5.019941	1	0	0	0.000000	5.374568e+22	1.142748			
6.067069		0	1.688830	0	6	1431	1	2	3.375573	34.80	11	4	1	
114584		2	1	1	4.732454	1	0	0	8.462045	5.563510e+22	1.134287			
6.067069		0	1.748484	1	5	1432	2	2	3.204406	59.90	11	2	1	

[101736 rows x 23 columns]

```
[ ]: report_outlier_percentage(dataset_cleared_new_copy, continuous_columns_3)
```

```
WB: 1.44%
DR: 0.00%
GL: 0.00%
DMP: 0.27%
DFS: 0.04%
DM: 0.00%
AM: 0.14%
```

```
[ ]: total_number = 114994 - len(dataset_cleared_new_copy)
calculate_total_percent = (total_number / 114994) * 100
print(f"total percent outliers which removed from dataset:
      ↪{calculate_total_percent}")
print("-----")
all_percent_of_outliers = 6.65 + 3.16 + 0.39 + 1.79 + 1.08
print(f"total percent of outliers calculated by IQR:{all_percent_of_outliers}")
```

```
total percent outliers which removed from dataset:11.529297180722473
```

```
-----
```

```
total percent of outliers calculated by IQR:13.070000000000002
```

```
[ ]: #plots the distribution diagrams for continuous columns after removing outliers
n_cols = 4
n_rows = 2
continuous_columns_after_outliers = ["WB", "DR", "GL", "DMP", "DFS", "AI",
      ↪"DM", "AM"]
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols*4, n_rows*3))
axes = axes.flatten()
colors_cont_2 = sns.color_palette("tab20",
      ↪len(continuous_columns_after_outliers))
for i, col in enumerate(continuous_columns_after_outliers):
    sns.histplot(dataset_cleared_new_copy[col], kde=True, bins=30,
      ↪color=colors_cont_2[i], ax=axes[i])
    axes[i].set_title(col, fontsize=12, fontweight="bold")
    axes[i].set_xlabel('')
    axes[i].set_ylabel("Count")
    axes[i].grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/
      ↪continuous_cols_distributions_after_outlier.png", dpi=900)
plt.close()
```

```
[ ]: categorical_cols_category = ["PAS", "BT", "SEX", "DS", "RP", "NPA",
      "TP", "CI", "TNR", "MI", "YI", "LP", "DI", "CL",
      ↪"PS"]
```

```
[ ]: cont_columns = ["WB", "DR", "GL", "DMP", "DFS", "AI", "DM", "AM"]
cat_columns = ["PAS", "BT", "SEX", "DS", "RP", "TNR", "NPA", "DI", "MI",
               "YI", "LP", "TP", "CL", "CI"]
target_col = ["PS"]
def plot_mi(df, continuous_cols, cat_cols, target, save_path=None):
    """
    This function calculates the mutual information value for each column
    then plots the diagram for it
    """
    all_features = continuous_cols + cat_cols
    X = df[all_features]
    Y = df[target]
    discrete_features = [False] * len(continuous_cols) + [True] * len(cat_cols)
    mi_values = mutual_info_classif(X, Y, discrete_features=discrete_features,
    ↪random_state=42)
    mi_table = pd.DataFrame({
        "Feature": all_features,
        "Mutual Information": mi_values
    }).sort_values(by="Mutual Information", ascending=False)
    plt.figure(figsize=(12, 7))
    sns.set(style="whitegrid")
    bar_colors = sns.color_palette("viridis", len(mi_table))
    bars = plt.barh(mi_table["Feature"], mi_table["Mutual Information"],
    ↪color=bar_colors)
    plt.xlabel("Mutual Information", fontsize=12)
    plt.ylabel("Features", fontsize=12)
    plt.title(f" Mutual Information Analysis of {target}", fontsize=16,
    ↪fontweight="bold")
    plt.gca().invert_yaxis()
    for bar, mi_val in zip(bars, mi_table["Mutual Information"]):
        plt.text(bar.get_width() * 1.01, bar.get_y() + bar.get_height()/2,
                 f"{mi_val:.4f}", va="center", fontsize=10)
    plt.tight_layout()
    if save_path:
        os.makedirs(save_path, exist_ok=True)
        plt.savefig(os.path.join(save_path, "mutual_information.png"), dpi=900,
    ↪bbox_inches="tight")
    plt.close()
    return mi_table

first_path = "/content/drive/MyDrive/project_first/"
mi_df = plot_mi(dataset_cleared_new_copy, cont_columns, cat_columns,
    ↪target_col, save_path=first_path)
print(mi_df)
```

Feature Mutual Information

1	DR	4.588281e-03
16	MI	3.353389e-03
0	WB	3.225144e-03
5	AI	2.863882e-03
6	DM	2.743486e-03
2	GL	2.249450e-03
18	LP	1.749512e-03
13	TNR	6.720195e-04
17	YI	4.747712e-04
11	DS	3.495461e-04
21	CI	2.030649e-04
19	TP	2.017180e-04
7	AM	1.780043e-04
12	RP	1.667567e-04
4	DFS	1.288772e-04
10	SEX	1.045187e-04
14	NPA	9.153286e-05
15	DI	6.877259e-05
8	PAS	5.281151e-05
20	CL	4.164013e-06
9	BT	4.523624e-08
3	DMP	0.000000e+00

```
[ ]: def cramers_v(x, y):
    """
    This function calculates the Cramer's v value
    for caegorical columns then plots heatmap for them
    """
    correlation_matrix = pd.crosstab(x, y)
    chi2 = chi2_contingency(correlation_matrix)[0]
    n = correlation_matrix.sum().sum()
    r, k = correlation_matrix.shape
    return np.sqrt(chi2 / (n * (min(k-1, r-1))))

def plot_cramers_v(df, cate_cols, tar_col, save_path=None):
    all_cols = cate_cols + [tar_col]
    cramers_matrix = pd.DataFrame(np.zeros((len(all_cols), len(all_cols))),
                                  index=all_cols, columns=all_cols)

    for col1 in all_cols:
        for col2 in all_cols:
            if col1 == col2:
                cramers_matrix.loc[col1, col2] = 1.0
            else:
                cramers_matrix.loc[col1, col2] = cramers_v(df[col1], df[col2])
    plt.figure(figsize=(12, 10))
    sns.set(style="white")
    cmap = sns.diverging_palette(220, 20, as_cmap=True)
```

```

ax = sns.heatmap(cramers_matrix, annot=True, fmt=".2f", cmap=cmap, cbar=True,
                  linewidths=0.5, linecolor="gray", square=True)
plt.title("Cramers V Correlation Matrix", fontsize=16, fontweight="bold")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
if save_path:
    os.makedirs(save_path, exist_ok=True)
    plt.savefig(os.path.join(save_path, "cramers_v_heatmap.png"), dpi=900,
↳bbox_inches="tight")
plt.close()
return crammers_matrix

cat_columns = ["PAS", "BT", "SEX", "DS", "RP", "TNR", "NPA", "DI", "MI",
               "YI", "LP", "TP", "CL", "CI"]
target_col = "PS"

cramers_mat = plot_cramers_v(dataset_cleared_new_copy, cat_columns, target_col,
↳save_path=first_path)
print(cramers_mat)

```

	PAS	BT	SEX	DS	RP	TNR	NPA
DI	MI	YI	LP	TP	CL	CI	PS
PAS	1.000000	0.006951	0.036335	0.209663	0.032248	0.040016	0.013740
	0.005510	0.009539	0.082284	0.103976	0.008253	0.002904	0.017241
BT	0.006951	1.000000	1.000000	0.008268	0.031148	0.038439	0.005228
	0.005099	0.010772	0.018350	0.023938	0.001417	0.002728	0.006106
SEX	0.036335	1.000000	1.000000	0.057174	0.047175	0.031396	0.008933
	0.006011	0.009581	0.026505	0.034694	0.005416	0.006658	0.008559
DS	0.209663	0.008268	0.057174	1.000000	0.395007	0.156992	0.014953
	0.003303	0.031520	0.327137	0.075061	0.019996	0.002456	0.055798
RP	0.032248	0.031148	0.047175	0.395007	1.000000	0.619520	0.031229
	0.006306	0.037904	0.569719	0.055078	0.021213	0.008935	0.102069
TNR	0.040016	0.038439	0.031396	0.156992	0.619520	1.000000	0.062680
	0.008302	0.018602	0.176956	0.310227	0.026472	0.008571	0.083145
NPA	0.013740	0.005228	0.008933	0.014953	0.031229	0.062680	1.000000
	0.007218	0.021814	0.052400	0.060071	0.011895	0.005585	0.021411
DI	0.005510	0.005099	0.006011	0.003303	0.006306	0.008302	0.007218
	1.000000	0.025430	0.016259	0.008826	0.005180	0.586588	0.006446
MI	0.009539	0.010772	0.009581	0.031520	0.037904	0.018602	0.021814
	0.025430	1.000000	0.083945	0.012071	0.024472	0.023597	0.037885
YI	0.082284	0.018350	0.026505	0.327137	0.569719	0.176956	0.052400
	0.016259	0.083945	1.000000	0.047886	0.087864	0.016883	0.238696
LP	0.103976	0.023938	0.034694	0.075061	0.055078	0.310227	0.060071
	0.008826	0.012071	0.047886	1.000000	0.022230	0.010762	0.020856
							0.059294

```

TP    0.008253  0.001417  0.005416  0.019996  0.021213  0.026472  0.011895
0.005180  0.024472  0.087864  0.022230  1.000000  0.004907  0.053094  0.019131
CL    0.002904  0.002728  0.006658  0.002456  0.008935  0.008571  0.005585
0.586588  0.023597  0.016883  0.010762  0.004907  1.000000  0.006657  0.002886
CI    0.017241  0.006106  0.008559  0.055798  0.102069  0.083145  0.021411
0.006446  0.037885  0.238696  0.020856  0.053094  0.006657  1.000000  0.020154
PS    0.010125  0.000057  0.014411  0.026220  0.018134  0.036419  0.013629
0.011737  0.081541  0.030922  0.059294  0.019131  0.002886  0.020154  1.000000

```

```
[ ]: dataset_cleared_new_copy.to_excel("/content/drive/MyDrive/project_first/
↳final_cleared_data.xlsx", index=False) #saves the cleared data
```

```
[ ]: file_path = "/content/drive/MyDrive/project_first/final_cleared_data.xlsx"
dataset_cleared_final = pd.read_excel(file_path) #displays cleared data
pd.set_option("display.max_columns",None) # for showing all columns
pd.set_option("display.expand_frame_repr",False) # to avoid data clutter
dataset_cleared_final
```

```
[ ]:
```

	PAS	BT	SEX		WB	DS	RP	TNR		DR		GL		DMP
DFS	NPA		AI	DI	MI	YI	LP	TP		DM	AM	CL	CI	PS
0		2	1	1	4.770944	2	1	9		8.202830	4.356831e+22	1.152087		
6.093963		0	1.930030	3	3	1425	10	2		4.352277	19.60	21	2	0
1		2	1	1	4.770944	2	1	9		8.202830	4.356831e+22	1.152087		
6.093963		0	1.930425	2	4	1425	10	2		4.386036	17.20	2	2	0
2		2	1	0	4.732454	2	1	8		11.121507	5.758387e+22	1.146627		
6.145662		1	1.927888	2	3	1425	10	2		3.466952	30.40	21	2	0
3		2	1	0	4.732454	2	0	7		9.958425	5.959364e+22	1.148228		
6.145662		0	1.926683	0	9	1425	11	2		3.507714	36.10	1	2	1
4		2	1	0	4.770944	2	1	10		8.398384	4.675000e+22	1.132001		
6.170537		0	1.920583	0	2	1425	10	2		3.993677	30.60	1	2	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
101731		2	1	1	4.881791	2	0	0		10.430736	5.374568e+22	1.145569		
6.085079		0	1.861694	3	4	1436	5	2		4.124313	24.80	1	2	0
101732		2	1	1	4.881791	2	0	0		10.430736	5.374568e+22	1.145569		
6.085079		0	1.862806	2	5	1436	5	2		4.181202	23.25	21	4	0
101733		2	1	1	4.881791	2	0	0		10.430736	5.374568e+22	1.145569		
6.085079		0	1.864987	1	7	1436	5	2		4.278707	25.10	11	2	0
101734		2	1	0	5.019941	1	0	0		0.000000	5.374568e+22	1.142748		
6.067069		0	1.688830	0	6	1431	1	2		3.375573	34.80	11	4	1
101735		2	1	1	4.732454	1	0	0		8.462045	5.563510e+22	1.134287		
6.067069		0	1.748484	1	5	1432	2	2		3.204406	59.90	11	2	1

[101736 rows x 23 columns]

```
[ ]: dataset_cleared_final_copy = dataset_cleared_final.copy()
```

```
[ ]: dataset_cleared_final_copy.drop(["CL", "BT", "DMP"], axis=1, inplace=True)
      ↪ #removes the columns CL,BT, and DMP

[ ]: target_column11 = "PS"
      high_card_col = ["TNR", "LP"]
      rare_threshold = 0.01
      def analyze_categorical(df, col, target_col_11, rare_threshold=rare_threshold):
          """
          This function analyzes a categorical column by displaying
          the distribution of its categories, identifying rare categories below
          a specified threshold, and computing the mean and standard deviation of
          the target column for each category
          """
          freq = df[col].value_counts(normalize=True)
          print(f"distribution of categories({len(freq)}): ")
          print(freq)
          rare_cats = freq[freq < rare_threshold].index
          print(f"rare categories({len(rare_cats)}): {list(rare_cats)}")
          target_mean = df.groupby(col)[target_col_11].mean()
          print("target mean for each category: ")
          print(target_mean)
          std_rate = target_mean.std()
          print(f"std rate of target column between categories: {std_rate:.4f}")

      for col in high_card_col:
          print(f"column:{col}")
          analyze_categorical(dataset_cleared_final_copy, col, "PS")
```

```
column:TNR
distribution of categories(11):
TNR
0    0.687770
1    0.186443
2    0.064589
3    0.031346
4    0.013987
5    0.007755
6    0.004315
7    0.002025
8    0.001189
9    0.000462
10   0.000118
Name: proportion, dtype: float64
rare categories(6): [5, 6, 7, 8, 9, 10]
target mean for each category:
TNR
```

```

0      0.313387
1      0.282159
2      0.275909
3      0.264346
4      0.288124
5      0.288973
6      0.250569
7      0.281553
8      0.206612
9      0.340426
10     0.416667
Name: PS, dtype: float64
std rate of target column between categories: 0.0535
column:LP
distribusion of categories(14):
LP
1      0.345532
2      0.271074
3      0.164878
4      0.101891
5      0.056804
6      0.030736
7      0.015766
8      0.007628
9      0.003499
10     0.001484
11     0.000550
13     0.000069
12     0.000069
14     0.000020
Name: proportion, dtype: float64
rare categories(7): [8, 9, 10, 11, 13, 12, 14]
target mean for each category:
LP
1      0.338634
2      0.289796
3      0.287409
4      0.279471
5      0.273577
6      0.260953
7      0.259975
8      0.257732
9      0.283708
10     0.238411
11     0.321429
12     0.428571
13     0.142857
14     0.500000

```


Name: PS, dtype: float64

std rate of target column between categories: 0.0848

```
[ ]: def plot_stacked_bar_target_rate(df, cat_col, target_col):  
    """  
    This function plots stacked bar chart for the TNR column  
    and LP columns to visually compare the distribution of categories and their  
    relation to the target variable  
    """  
  
    prop_df = (  
        df.groupby(cat_col)[target_col]  
        .value_counts(normalize=True)  
        .rename("proportion")  
        .reset_index()  
        .pivot(index=cat_col, columns=target_col, values="proportion")  
        .fillna(0)  
    )  
  
    prop_df = prop_df.sort_index()  
  
    fig, ax = plt.subplots(figsize=(9, 6))  
    bottom = np.zeros(len(prop_df))  
    colors = ["#5DADE2", "#48C9B0"]  
  
    for i, col in enumerate(prop_df.columns):  
        bars = ax.bar(  
            prop_df.index.astype(str),  
            prop_df[col],  
            bottom=bottom,  
            color=colors[i],  
            edgecolor="black",  
            width=0.7,  
            label=f"{target_col} = {col}",  
        )  
  
        for bar, value, base in zip(bars, prop_df[col], bottom):  
            if value > 0.02:  
                ax.text(  
                    bar.get_x() + bar.get_width() / 2,  
                    base + value / 2,  
                    f"{value * 100:.1f}%",  
                    ha="center",  
                    va="center",  
                    fontsize=9,
```

```

        color="white",
        fontweight="bold",
    )
    bottom += prop_df[col].values

    ax.set_title(f"Distribution of {target_col} within {cat_col}", fontsize=13,
↳pad=12)
    ax.set_xlabel(cat_col, fontsize=11)
    ax.set_ylabel("Proportion", fontsize=11)
    ax.set_ylim(0, 1.05)
    ax.legend(title="Target", loc="upper right", fontsize=9)
    ax.grid(axis="y", linestyle="--", alpha=0.6)
    plt.tight_layout()
    plt.savefig(f"/content/drive/MyDrive/project_first/stacked_plt_{cat_col}.
↳png", dpi=600)
    plt.close()

plot_stacked_bar_target_rate(dataset_cleared_final_copy, "TNR", "PS")
plot_stacked_bar_target_rate(dataset_cleared_final_copy, "LP", "PS")

```

```

[ ]: def cluster_tnr(df, cat_col, target_col, method="ward", metric="euclidean",
↳min_samples=50):
    """
    This function plots dendrogram diagram
    """

    summary = df.groupby(cat_col)[target_col].mean().reset_index()
    linked = linkage(summary[[target_col]], method=method, metric=metric)
    sns.set_theme(style="whitegrid")
    rcParams["font.family"] = "DejaVu Sans"
    rcParams["axes.titlesize"] = 16
    rcParams["axes.labelsize"] = 13
    rcParams["xtick.labelsize"] = 10
    rcParams["ytick.labelsize"] = 10
    plt.figure(figsize=(10, 6), dpi=900)
    dend = dendrogram(linked, labels=summary[cat_col].astype(str).values,
        leaf_rotation=0, leaf_font_size=10,
↳above_threshold_color="gray", color_threshold=None, orientation="top")
    ax = plt.gca()
    for i, line in enumerate(ax.get_lines()):
        line.set_color(sns.color_palette("tab20")[i % 10])
        line.set_linewidth(2.2)
    plt.title(f"Dendrogram of {cat_col} Based on Target Mean {target_col}",
↳fontsize=14, fontweight="bold")

```

```

plt.xlabel(f"Categories of {cat_col}", fontsize=12)
plt.ylabel("Distance", fontsize=12)
plt.grid(alpha=0.3, linestyle="--")
legends_labels = [f"Cluster {i+1}" for i in
↪range(len(set(dend["color_list"])))])
color = list(set(dend["color_list"]))
plt.legend(handles=[plt.Line2D([0], [0], color=c, lw=3) for c in color],
          labels=legends_labels, loc="upper right", fontsize=10,
↪frameon=True)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/dendrogram.png", dpi=900)
plt.close()

```

```
summary = cluster_tnr(dataset_cleared_final_copy, "TNR", "PS")
```

```

[ ]: def apply_manual_merge(df, cat_col, mapping_dict, new_col=None,
                             keep_unmapped_as_original=True, unmapped_value="other",
↪target_col=None, verbose=True):
    """
    This function merges close categories for TNR column
    and makes new column for it's result
    """
    if new_col is None:
        new_col = f"{cat_col}_merged"
    mapping_str = {str(k): v for k, v in mapping_dict.items()}
    col_as_str = df[cat_col].astype(str)
    mapped = col_as_str.map(mapping_str)
    if keep_unmapped_as_original:
        result = mapped.fillna(col_as_str)
    else:
        result = mapped.fillna(unmapped_value)
    df[new_col] = result
    if verbose and target_col is not None:
        print(" == AFTER MERGE ==")
        print("Value counts (merged):")
        print(df[new_col].value_counts)
        print("\nTarget mean per merged category:")
        print(df.groupby(new_col)[target_col].mean())
        print("\nMapping applied (sample):")
        applied = pd.Series(mapping_str).reset_index()
        applied.columns = ["orig_key", "mapped_to"]
        print(applied.head(15))
    return df

```

```
mapping_tnr = {
```

```

    10:"group1", 8:"group1",
    0:"group2", 9:"group2",
    4:"group3", 5:"group3", 2:"group3", 1:"group3", 7:"group3",
    3:"group4", 6:"group4"
}
dataset_cleared_final_copy = apply_manual_merge(dataset_cleared_final_copy,
↳cat_col="TNR",
mapping_dict=mapping_tnr,
↳new_col="TNR_MERGED",
keep_unmapped_as_original=True,
↳target_col="PS", verbose=True)

```

```

== AFTER MERGE ==
Value counts (merged):
<bound method IndexOpsMixin.value_counts of 0          group2
1          group2
2          group1
3          group3
4          group1
...
101731     group2
101732     group2
101733     group2
101734     group2
101735     group2
Name: TNR_MERGED, Length: 101736, dtype: object>

```

Target mean per merged category:

```

TNR_MERGED
group1    0.225564
group2    0.313405
group3    0.281182
group4    0.262679
Name: PS, dtype: float64

```

Mapping applied (sample):

```

    orig_key mapped_to
0         10   group1
1          8   group1
2          0   group2
3          9   group2
4          4   group3
5          5   group3
6          2   group3
7          1   group3
8          7   group3
9          3   group4

```

10 6 group4

```
[ ]: merge_map = {
    "group1":0,
    "group2":1,
    "group3":2,
    "group4":3
}
dataset_cleared_final_copy["TNR_MERGED"] =
    ↪dataset_cleared_final_copy["TNR_MERGED"].map(merge_map)
```

```
[ ]: plot_stacked_bar_target_rate(dataset_cleared_final_copy, "TNR_MERGED", "PS")
```

```
[ ]: #plots the distribution diagram(countplot) for the TNR_MERGED column
sorted_categories = sorted(dataset_cleared_final_copy["TNR_MERGED"].unique())
plt.figure(figsize=(10, 8), dpi=900)
ax = sns.countplot(data=dataset_cleared_final_copy, x="TNR_MERGED",
    ↪order=sorted_categories, palette="Set2", edgecolor="black")
for p in ax.patches:
    height = p.get_height()
    ax.annotate(f"{int(height)}", (p.get_x() + p.get_width() / 2, height),
    ↪ha="center", va="center", fontsize=10)
plt.ylabel("Frequency", fontsize=12)
plt.xlabel("TNR_MERGED", fontsize=12)
plt.title("COUNT Plot of TNR_MERGED Categories", fontsize=14, fontweight="bold")
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/distribution_of_TNRMERGED.
    ↪png", dpi=900)
plt.close()
```

```
[ ]: # performs one-hot encoding for LP column
df_lp_encoded = pd.get_dummies(dataset_cleared_final_copy["LP"], prefix="LP",
    ↪drop_first=False, dtype=int)
dataset_cleared_final_copy = pd.concat([dataset_cleared_final_copy,
    ↪df_lp_encoded], axis=1)
dataset_cleared_final_copy
```

```
[ ]:
```

	PAS	SEX		WB	DS	RP	TNR		DR		GL	DFS	NPA	
AI	DI	MI	YI	LP	TP		DM	AM	CI	PS	TNR_MERGED	LP_1	LP_2	LP_3
LP_4	LP_5	LP_6	LP_7	LP_8	LP_9	LP_10	LP_11	LP_12	LP_13	LP_14				
0		2	1	4.770944	2	1	9	8.202830	4.356831e+22	6.093963	0			
1.930030	3	3	1425	10	2	4.352277	19.60	2	0		1	0	0	
0	0	0	0	0	0	0	1	0	0	0	0	0		
1		2	1	4.770944	2	1	9	8.202830	4.356831e+22	6.093963	0			
1.930425	2	4	1425	10	2	4.386036	17.20	2	0		1	0	0	
0	0	0	0	0	0	0	1	0	0	0	0	0		

2	2	0	4.732454	2	1	8	11.121507	5.758387e+22	6.145662	1
1.927888	2	3	1425	10	2	3.466952	30.40	2	0	0
0	0	0	0	0	0	0	1	0	0	0
3	2	0	4.732454	2	0	7	9.958425	5.959364e+22	6.145662	0
1.926683	0	9	1425	11	2	3.507714	36.10	2	1	0
0	0	0	0	0	0	0	0	1	0	0
4	2	0	4.770944	2	1	10	8.398384	4.675000e+22	6.170537	0
1.920583	0	2	1425	10	2	3.993677	30.60	2	0	0
0	0	0	0	0	0	0	1	0	0	0
...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...
101731	2	1	4.881791	2	0	0	10.430736	5.374568e+22	6.085079	0
1.861694	3	4	1436	5	2	4.124313	24.80	2	0	0
0	0	1	0	0	0	0	0	0	0	0
101732	2	1	4.881791	2	0	0	10.430736	5.374568e+22	6.085079	0
1.862806	2	5	1436	5	2	4.181202	23.25	4	0	0
0	0	1	0	0	0	0	0	0	0	0
101733	2	1	4.881791	2	0	0	10.430736	5.374568e+22	6.085079	0
1.864987	1	7	1436	5	2	4.278707	25.10	2	0	0
0	0	1	0	0	0	0	0	0	0	0
101734	2	0	5.019941	1	0	0	0.000000	5.374568e+22	6.067069	0
1.688830	0	6	1431	1	2	3.375573	34.80	4	1	0
0	0	0	0	0	0	0	0	0	0	0
101735	2	1	4.732454	1	0	0	8.462045	5.563510e+22	6.067069	0
1.748484	1	5	1432	2	2	3.204406	59.90	2	1	1
0	0	0	0	0	0	0	0	0	0	0

[101736 rows x 35 columns]

```
[ ]: dataset_cleared_final_copy = dataset_cleared_final_copy.drop(columns=["LP"])
```

```
[ ]: cont_columns_11 = ["WB", "DR", "GL", "DFS", "AI", "DM", "AM"]
```

```
[ ]: #Performs scaling for continuous columns
scaler = MinMaxScaler()
dataset_cleared_final_copy[cont_columns_11] = scaler.
    ↪fit_transform(dataset_cleared_final_copy[cont_columns_11])
```

#SMOTE TECHNIQUE it is required due to imbalance data in target column

```
[ ]: X = dataset_cleared_final_copy.drop(columns=["PS"])
```

```
Y = dataset_cleared_final_copy["PS"]
```

```
[ ]: #performs the SMOTENC method for balancing the data
cat_idx = [0, 1, 3, 4, 5, 9, 11, 12, 13, 14, 17, 19, 20, 21, 22, 23, 24, 25,
    ↪26, 27, 28, 29, 30, 31, 32]
```

```
smote_nc = SMOTENC(categorical_features=cat_idx, random_state=42)
X_resmapled, y_resampled = smote_nc.fit_resample(X, Y)
```

```
[ ]: df_final = pd.concat([X_resmapled, y_resampled.rename("PS")], axis=1)
df_final
```

```
[ ]:
```

	PAS	SEX		WB	DS	RP	TNR		DR		GL		DFS	NPA
AI	DI	MI	YI	TP		DM		AM	CI	TNR_MERGED	LP_1	LP_2	LP_3	LP_4
LP_5	LP_6	LP_7	LP_8	LP_9	LP_10	LP_11	LP_12	LP_13	LP_14	PS				
0		2	1	0.394371	2	1	9	0.409464	0.338761	0.550996	0			
0.966327	3	3	1425	2	0.768197	0.115212	2		1	0	0	0		
0	0	0	0	0	0	1	0	0	0	0	0	0		
1		2	1	0.394371	2	1	9	0.409464	0.338761	0.550996	0			
0.967649	2	4	1425	2	0.785543	0.072007	2		1	0	0	0	0	
0	0	0	0	0	0	1	0	0	0	0	0	0		
2		2	0	0.354425	2	1	8	0.555157	0.628844	0.695246	1			
0.959152	2	3	1425	2	0.313296	0.309631	2		0	0	0	0	0	
0	0	0	0	0	0	1	0	0	0	0	0	0		
3		2	0	0.354425	2	0	7	0.497099	0.670440	0.695246	0			
0.955120	0	9	1425	2	0.334240	0.412241	2		2	0	0	0	0	
0	0	0	0	0	0	0	1	0	0	0	1			
4		2	0	0.394371	2	1	10	0.419226	0.404613	0.764653	0			
0.934694	0	2	1425	2	0.583939	0.313231	2		0	0	0	0	0	
0	0	0	0	0	0	1	0	0	0	0	0	0		
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
141891	2	0	0.401914	1	0	0	0.364731	0.551968	0.463046	0				
0.496499	1	3	1433	2	0.393434	0.548435	2		1	0	0	0	1	
0	0	0	0	0	0	0	0	0	0	0	1			
141892	2	0	0.743560	1	0	0	0.000000	0.667319	0.598358	1				
0.335937	3	8	1428	2	0.777258	0.216113	2		1	1	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	1			
141893	2	0	0.425158	1	1	1	0.428652	0.757418	0.354700	0				
0.333044	1	2	1437	2	0.343159	0.667127	3		2	0	1	0	0	
0	0	0	0	0	0	0	0	0	0	0	1			
141894	2	0	0.657286	1	1	2	0.638262	0.549404	0.596274	0				
0.534491	1	2	1428	2	0.488965	0.586820	2		2	0	0	1	0	
0	0	0	0	0	0	0	0	0	0	0	1			
141895	2	1	0.229253	2	1	4	0.436364	0.547479	0.840211	0				
0.735526	1	8	1425	2	0.871617	0.064264	2		2	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	0	1			

[141896 rows x 34 columns]

```
[ ]: # shuffles data
df_final = df_final.sample(frac=1, random_state=42).reset_index(drop=True)
```

```
df_final
```

```
[ ]:      PAS SEX      WB DS RP TNR      DR      GL      DFS NPA
AI DI MI  YI TP      DM      AM CI TNR_MERGED LP_1 LP_2 LP_3 LP_4
LP_5 LP_6 LP_7 LP_8 LP_9 LP_10 LP_11 LP_12 LP_13 LP_14 PS
0      2  0  0.720676  2  0  0  0.000000  1.000000  0.963023  0
0.190558  3  2  1431  2  0.382554  0.642664  2      1  1  0  0
0  0  0  0  0  0  0  0  0  0  0  0
1      2  0  0.582434  1  0  0  0.381536  0.713334  0.538629  0
0.483261  3  1  1432  2  0.245275  0.514851  1      1  0  0  1
0  0  0  0  0  0  0  0  0  0  0  0
2      2  0  0.849743  1  0  0  0.440743  0.628844  0.695246  0
0.364178  2  12  1430  2  0.491993  0.545455  3      1  0  1  0
0  0  0  0  0  0  0  0  0  0  0  1
3      2  1  0.313618  1  0  0  0.000000  0.338761  0.326197  0
0.125261  1  7  1436  2  0.443925  0.473447  4      1  1  0  0
0  0  0  0  0  0  0  0  0  0  0  1
4      2  0  0.332038  1  0  0  0.000000  0.439129  0.322908  0
0.104692  0  11  1437  2  0.417677  0.533222  2      1  1  0  0
0  0  0  0  0  0  0  0  0  0  0  1
...  ...  ...  ...  ..  ..  ...  ...  ...  ...  ...
...  ..  ..  ...  ..  ...  ...  ..  ...  ...  ...
...  ...  ...  ...  ...  ...  ...  ...  ...  ...
141891  2  0  0.451533  1  0  0  0.000000  0.385988  0.337820  0
0.093550  1  3  1436  2  0.370694  0.658013  2      1  1  0  0
0  0  0  0  0  0  0  0  0  0  0  1
141892  2  0  0.342431  1  0  0  0.000000  0.474746  0.331358  0
0.107254  0  5  1433  2  0.420290  0.406831  2      1  1  0  0
0  0  0  0  0  0  0  0  0  0  0  1
141893  2  0  0.415364  1  0  0  0.000000  0.648217  0.382927  0
0.139037  3  6  1434  2  0.464215  0.575014  2      1  1  0  0
0  0  0  0  0  0  0  0  0  0  0  1
141894  2  1  0.483998  1  0  0  0.352977  0.524636  0.513730  1
0.669205  3  12  1430  2  0.262806  0.635943  3      1  0  0  0
1  0  0  0  0  0  0  0  0  0  0  1
141895  2  0  0.316283  1  0  0  0.410515  0.546927  0.618828  0
0.640688  2  1  1433  2  0.520271  0.706964  2      1  0  0  0
1  0  0  0  0  0  0  0  0  0  0  1

[141896 rows x 34 columns]
```

```
[ ]: #prepares settings for pycaret
target_col_pycaret = "PS"
numeric_features = ["WB", "DR", "GL", "DFS", "AI", "DM", "AM"]
categorical_features = ["PAS", "SEX", "DS", "RP", "TNR", "NPA", "DI", "MI",
    ↪ "YI", "TP", "CI", "TNR_MERGED",
    "LP_1", "LP_2", "LP_3", "LP_4", "LP_5", "LP_6",
```



```

        "LP_7", "LP_8", "LP_9", "LP_10", "LP_11", "LP_13",
        ↪ "LP_14"]
s = setup(data=df_final, target=target_col_pycaret,
        ↪ numeric_features=numeric_features, categorical_features=categorical_features,
            preprocess=False, normalize=False, polynomial_features=False,
        ↪ remove_outliers=False, fold_shuffle=True,
            session_id=42, verbose=True, train_size=0.8)

```

<pandas.io.formats.style.Styler at 0x798e40dc5fa0>

```

[ ]: best_models = compare_models(n_select=8, sort="Accuracy") # pycaret trains and
        ↪ test models then chooses best 8 models

```

<IPython.core.display.HTML object>

<pandas.io.formats.style.Styler at 0x798e46afef60>

Processing: 0% | 0/72 [00:00<?, ?it/s]

<IPython.core.display.HTML object>

```

[ ]: results_df = pull() # saves pycaret's results
results_df.to_excel("/content/drive/MyDrive/project_first/
        ↪ pycaret_models_results.xlsx", index=False)

```

```

[ ]: # saves parameters which pycaret used for each models
params_list = []
for model in best_models:
    model_name = str(model).split("(")[0]
    params = model.get_params()
    params_list.append({
        "model": model_name,
        "parameters": params
    })
params_df = pd.DataFrame(params_list)
params_df.to_excel("/content/drive/MyDrive/project_first/pycaret_models_params.
        ↪ xlsx", index=False)

```

```

[ ]: #plots confusin matrix for pycaret results
x_test = get_config("X_test")
y_test = get_config("y_test")
fig, ax = plt.subplots(2, 4, figsize=(20, 10), dpi=900)
ax = ax.flatten()
for i, model in enumerate(best_models):
    preds = predict_model(model, data=x_test)
    y_pred = preds["prediction_label"]
    cm = confusion_matrix(y_test, y_pred)
    labels = np.unique(y_test)

```

```

sns.heatmap(cm, annot=True, fmt="d", cmap="YlGnBu", cbar=False,
xticklabels=labels, yticklabels=labels, ax=ax[i])
ax[i].set_title(str(model).split('(')[0], fontsize=10)
ax[i].set_xlabel("Predicted", fontsize=8)
ax[i].set_ylabel("Actual", fontsize=8)
ax[i].tick_params(axis="both", labelsize=8)
for j in range(len(best_models), len(ax)):
    fig.delaxes(ax[j])
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/confusion_matrix.png",
dpi=900)
plt.close()

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```

[ ]: my_df_results = pd.read_excel("/content/drive/MyDrive/project_first/
pycaret_models_results.xlsx")
my_df_results

```

```

[ ]:

```

			Model	Accuracy	AUC	Recall	Prec.	F1
Kappa	MCC	TT (Sec)						
0	Light Gradient Boosting Machine			0.7712	0.8204	0.5656	0.9606	0.7120
0.5424	0.5950	83.348						
1	Random Forest Classifier			0.7577	0.8374	0.7282	0.7739	0.7503
0.5154	0.5163	5.189						
2	Extreme Gradient Boosting			0.7522	0.8130	0.5822	0.8823	0.7015
0.5045	0.5365	0.554						
3	Extra Trees Classifier			0.7501	0.8320	0.7459	0.7523	0.7490
0.5002	0.5003	4.281						
4	Gradient Boosting Classifier			0.7367	0.7943	0.5911	0.8339	0.6918
0.4733	0.4948	6.451						
5	Decision Tree Classifier			0.6605	0.6605	0.6617	0.6601	0.6609
0.3210	0.3210	0.571						
6	K Neighbors Classifier			0.6330	0.6800	0.7121	0.6148	0.6599
0.2659	0.2693	2.995						
7	Ada Boost Classifier			0.6215	0.6730	0.6322	0.6192	0.6255
0.2430	0.2432	1.410						

8		Ridge Classifier	0.5749	0.6037	0.6127	0.5696	0.5904
0.1497	0.1502	0.037					
9		Linear Discriminant Analysis	0.5749	0.6037	0.6126	0.5696	0.5903
0.1497	0.1502	0.095					
10		Logistic Regression	0.5661	0.5912	0.6125	0.5605	0.5853
0.1321	0.1327	3.232					
11		Naive Bayes	0.5255	0.5744	0.8980	0.5161	0.6524
0.0510	0.0800	0.039					
12		Quadratic Discriminant Analysis	0.5093	0.6000	0.1865	0.8943	0.1414
0.0185	0.0628	0.371					
13		SVM - Linear Kernel	0.5056	0.5743	0.6885	0.4739	0.4710
0.0112	0.0309	2.574					
14		Dummy Classifier	0.5000	0.5000	0.2000	0.1000	0.1333
0.0000	0.0000	0.035					

#PYCARET TOP MODELS: 1-Light Gradient Boosting Machine

2-Random Forest Classifier

3-Extreme Gradient Boosting

4-Extra Trees Classifier

5-Gradient Boosting Classifier

6-Decision Tree Classifier

7-K Neighbors Classifier

8-Ada Boost Classifier

```
[ ]: my_df_params = pd.read_excel("/content/drive/MyDrive/project_first/
    ↳pycaret_models_params.xlsx")
my_df_params
```

```
[ ]:
    model \
0      LGBMClassifier
1      RandomForestClassifier
2      XGBClassifier
3      ExtraTreesClassifier
4      GradientBoostingClassifier
5      DecisionTreeClassifier
6      KNeighborsClassifier
7      AdaBoostClassifier

    parameters
0 {'boosting_type': 'gbdt', 'class_weight': None...
1 {'bootstrap': True, 'ccp_alpha': 0.0, 'class_w...
2 {'objective': 'binary:logistic', 'base_score':...
3 {'bootstrap': False, 'ccp_alpha': 0.0, 'class_...
4 {'ccp_alpha': 0.0, 'criterion': 'friedman_mse'...
```

```

5 {'ccp_alpha': 0.0, 'class_weight': None, 'crit...
6 {'algorithm': 'auto', 'leaf_size': 30, 'metric...
7 {'algorithm': 'SAMME.R', 'estimator': None, 'l...

```

```

[ ]: # sets all parameters pycaret used for each model again
light_gbm = LGBMClassifier(boosting_type= 'gbdt', class_weight=None,
                           colsample_bytree=1.0, importance_type='split',
                           learning_rate=0.1, max_depth= -1,
                           ↪min_child_samples=20,
                           min_child_weight=0.001, min_split_gain=0.0,
                           ↪n_estimators=100,
                           n_jobs=-1, num_leaves=31, objective=None,
                           ↪random_state=42,
                           reg_alpha=0.0, reg_lambda=0.0,
                           subsample=1.0, subsample_for_bin=200000,
                           ↪subsample_freq=0)

rf = RandomForestClassifier(bootstrap=True, ccp_alpha=0.0, class_weight=None,
                           ↪criterion='gini', max_depth=None,
                           max_features='sqrt', max_leaf_nodes=None,
                           ↪max_samples=None, min_impurity_decrease=0.0,
                           min_samples_leaf=1, min_samples_split=2,
                           ↪min_weight_fraction_leaf=0.0,
                           monotonic_cst=None, n_estimators=100, n_jobs=-1,
                           ↪oob_score=False, random_state=42,
                           verbose=0, warm_start=False)

xgb = XGBRFClassifier(objective='binary:logistic', base_score=None,
                     ↪booster='gbtree', device='cpu', tree_method='auto', verbosity=0)
extr = ExtraTreeClassifier(ccp_alpha=0.0, criterion='gini', max_features='sqrt',
                          min_samples_leaf=1, min_samples_split=2,
                          ↪min_weight_fraction_leaf=0.0,
                          random_state=42)
gb = GradientBoostingClassifier(ccp_alpha=0.0, criterion='friedman_mse',
                               ↪learning_rate=0.1, loss='log_loss', max_depth=3,
                               min_impurity_decrease=0.0, min_samples_leaf=1,
                               ↪min_samples_split=2,
                               min_weight_fraction_leaf=0.0, n_estimators=100,
                               ↪random_state=42, subsample=1.0,
                               tol=0.0001, validation_fraction=0.1, verbose=0,
                               ↪warm_start=False)
dt = DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None, criterion='gini',
                           ↪min_impurity_decrease=0.0,
                           min_samples_leaf=1, min_samples_split=2,
                           ↪min_weight_fraction_leaf=0.0, random_state=42, splitter='best')
knn = KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',

```

```

metric_params=None, n_jobs=-1, n_neighbors=5, p=2,
weights='uniform')
ada = AdaBoostClassifier(algorithm='SAMME.R', estimator=None, learning_rate=1.
0, n_estimators=50, random_state=42)

```

```

[ ]: # uses same models and parameters for training and testing hybrid models
#first 4 best models sent to voting classifier
# the result of voting classifier and reset of 4 models sent to stacking
classifier
hybrid1 = VotingClassifier(estimators=[("LIGHTGBM", light_gbm), ("RF", rf),
("EXGBOOST", xgb), ("EXTR", extr)], voting="soft")
hybrid2 = StackingClassifier(estimators=[("voting", hybrid1), ("GB", gb),
("DT", dt), ("KNN", knn), ("ADA", ada)],
final_estimator=gb, cv=10)
start_time = time.time()
hybrid2.fit(get_config("X_train"), get_config("y_train"))
end_time = time.time()
training_time = end_time - start_time

```

```

[LightGBM] [Info] Number of positive: 56758, number of negative: 56758
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.019103 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1881
[LightGBM] [Info] Number of data points in the train set: 113516, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 51082, number of negative: 51082
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006670 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1881
[LightGBM] [Info] Number of data points in the train set: 102164, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 51082, number of negative: 51082
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.007796 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1880
[LightGBM] [Info] Number of data points in the train set: 102164, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 51082, number of negative: 51082

```

```

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006171 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1881
[LightGBM] [Info] Number of data points in the train set: 102164, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 51082, number of negative: 51082
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006929 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1880
[LightGBM] [Info] Number of data points in the train set: 102164, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 51082, number of negative: 51082
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006118 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1881
[LightGBM] [Info] Number of data points in the train set: 102164, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 51082, number of negative: 51082
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006229 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1882
[LightGBM] [Info] Number of data points in the train set: 102164, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 51082, number of negative: 51083
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006971 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1881
[LightGBM] [Info] Number of data points in the train set: 102165, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.499995 -> initscore=-0.000020
[LightGBM] [Info] Start training from score -0.000020
[LightGBM] [Info] Number of positive: 51082, number of negative: 51083
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006567 seconds.

```

You can set `force\_row\_wise=true` to remove the overhead.
And if memory is not enough, you can set `force\_col\_wise=true`.

```
[LightGBM] [Info] Total Bins 1880
[LightGBM] [Info] Number of data points in the train set: 102165, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.499995 -> initscore=-0.000020
[LightGBM] [Info] Start training from score -0.000020
[LightGBM] [Info] Number of positive: 51083, number of negative: 51082
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006567 seconds.
```

You can set `force\_row\_wise=true` to remove the overhead.
And if memory is not enough, you can set `force\_col\_wise=true`.

```
[LightGBM] [Info] Total Bins 1881
[LightGBM] [Info] Number of data points in the train set: 102165, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500005 -> initscore=0.000020
[LightGBM] [Info] Start training from score 0.000020
[LightGBM] [Info] Number of positive: 51083, number of negative: 51082
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.006912 seconds.
```

You can set `force\_row\_wise=true` to remove the overhead.
And if memory is not enough, you can set `force\_col\_wise=true`.

```
[LightGBM] [Info] Total Bins 1881
[LightGBM] [Info] Number of data points in the train set: 102165, number of used
features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500005 -> initscore=0.000020
[LightGBM] [Info] Start training from score 0.000020
```

```
[ ]: # saves all results
preds = hybrid2.predict(get_config("X_test"))
probs = hybrid2.predict_proba(get_config("X_test"))[:, 1]
y_true = get_config("y_test")
F1 = f1_score(y_true, preds, average="binary")
Accuracy = accuracy_score(y_true, preds)
AUC = roc_auc_score(y_true, probs)
Recall = recall_score(y_true, preds, average="binary")
Precision = precision_score(y_true, preds, average="binary")
MCC = matthews_corrcoef(y_true, preds)
Kappa = cohen_kappa_score(y_true, preds)
metrics = {
    "Model": "STACKING_VOTING_HYBRID ",
    "Accuracy": Accuracy,
    "AUC": AUC,
    "Recall": Recall,
    "Prec.": Precision,
    "F1": F1,
    "Kappa": Kappa,
```

```

        "MCC":MCC,
        "TT (Sec)":training_time
    }
    metrics_data = pd.DataFrame([metrics])
    df_all_models = pd.concat([my_df_results, metrics_data], ignore_index=True)
    df_all_models.to_excel("/content/drive/MyDrive/project_first/df_all_models.
↪xlsx", index=False)

```

```

[ ]: df_all = pd.read_excel("/content/drive/MyDrive/project_first/df_all_models.
↪xlsx")
df_all

```

```

[ ]:

```

	F1	Kappa	MCC	Model	Accuracy	AUC	Recall	Prec.
0	0.712000	0.542400	0.595000	Light Gradient Boosting Machine	0.771200	0.820400	0.565600	0.960600
1	0.750300	0.515400	0.516300	Random Forest Classifier	0.757700	0.837400	0.728200	0.773900
2	0.701500	0.504500	0.536500	Extreme Gradient Boosting	0.752200	0.813000	0.582200	0.882300
3	0.749000	0.500200	0.500300	Extra Trees Classifier	0.750100	0.832000	0.745900	0.752300
4	0.691800	0.473300	0.494800	Gradient Boosting Classifier	0.736700	0.794300	0.591100	0.833900
5	0.660900	0.321000	0.321000	Decision Tree Classifier	0.660500	0.660500	0.661700	0.660100
6	0.659900	0.265900	0.269300	K Neighbors Classifier	0.633000	0.680000	0.712100	0.614800
7	0.625500	0.243000	0.243200	Ada Boost Classifier	0.621500	0.673000	0.632200	0.619200
8	0.590400	0.149700	0.150200	Ridge Classifier	0.574900	0.603700	0.612700	0.569600
9	0.590300	0.149700	0.150200	Linear Discriminant Analysis	0.574900	0.603700	0.612600	0.569600
10	0.585300	0.132100	0.132700	Logistic Regression	0.566100	0.591200	0.612500	0.560500
11	0.652400	0.051000	0.080000	Naive Bayes	0.525500	0.574400	0.898000	0.516100
12	0.141400	0.018500	0.062800	Quadratic Discriminant Analysis	0.509300	0.600000	0.186500	0.894300
13	0.471000	0.011200	0.030900	SVM - Linear Kernel	0.505600	0.574300	0.688500	0.473900
14	0.133300	0.000000	0.000000	Dummy Classifier	0.500000	0.500000	0.200000	0.100000
15	0.750672	0.568428	0.590193	STACKING_VOTING_HYBRID	0.784214	0.854598	0.649683	0.888835


```
[ ]: # plots confusion matrix for hybrid model
cm = confusion_matrix(y_true, preds)
labels_h = np.unique(y_true)
plt.figure(figsize=(6, 5), dpi=900)
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues", cbar=False,
            xticklabels=labels_h, yticklabels=labels_h)
plt.title("Confusion Matrix - Hybrid Stacking", fontsize=14)
plt.xlabel("Predicted", fontsize=12)
plt.ylabel("Actual", fontsize=12)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/Hybrid_confusion_matrix.png",
            dpi=900)
plt.close()
```

```
[ ]: #plots radar diagram
top8_names = df_all["Model"].tolist()[0:8]
hybrid_name = df_all["Model"].iloc[-1]
models_plot = top8_names + [hybrid_name]
df_plot = df_all[df_all["Model"].isin(models_plot)].reset_index(drop=True)
metrics = ["Recall", "Prec.", "F1", "Kappa", "MCC"]
N = len(metrics)
angles = [n / float(N) * 2 * pi for n in range(N)]
angles += angles[:1]
plt.figure(figsize=(10, 10), dpi=900)
ax = plt.subplot(111, polar=True)
for i, row in df_plot.iterrows():
    values = row[metrics].tolist()
    values += values[:1]
    ax.plot(angles, values, linewidth=2, label=row["Model"])
    ax.fill(angles, values, alpha=0.1)
ax.set_theta_offset(pi / 2)
ax.set_theta_direction(-1)
plt.xticks(angles[:-1], metrics, fontsize=12)
ax.set_rlabel_position(0)
plt.yticks([0.2, 0.4, 0.6, 0.8, 1.0], ["0.2", "0.4", "0.6", "0.8", "1.0"],
            color="gray", size=10)
plt.ylim(0,1)
plt.title("Comparison of Models- Radar chart", fontsize=16)
plt.legend(loc="upper right", bbox_to_anchor=(1.3, 1.1))
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/Radar_chart.png", dpi=900)
plt.close()
```

```
[ ]: # plots barchart
df_plot_sorted = df_plot.sort_values("Accuracy", ascending=False).
            reset_index(drop=True)
plt.figure(figsize=(12, 6), dpi=900)
```

```

sns.set_style("whitegrid")
palette = sns.color_palette("tab10", n_colors=len(df_plot_sorted))
ax = sns.barplot(x="Model", y="Accuracy", data=df_plot_sorted, palette=palette)
for p in ax.patches:
    height = p.get_height()
    ax.text(
        x=p.get_x()+p.get_width()/2,
        y=height+0.005,
        s=f"{height:.3f}",
        ha="center",
        fontsize=10,
        weight="bold"
    )
plt.xticks(rotation=45, ha="right", fontsize=12)
plt.ylabel("Accuracy", fontsize=12, weight="bold")
plt.xlabel("Model", fontsize=12, weight="bold")
plt.title("Comparison of Model Accuracy", fontsize=16, weight="bold")
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/bar_plots_1.png", dpi=900)
plt.close()

```

```

[ ]: # prints the system's information used for the projects
metrics_1 = ["Accuracy", "AUC", "Recall", "Prec.", "F1", "Kappa", "MCC", "TT_
↳(Sec)"]
plt.figure(figsize=(14, len(df_all)*0.5+4), dpi=900)
sns.set_style("whitegrid")
cmap = sns.color_palette("coolwarm", as_cmap=True)
ax = sns.heatmap(
    df_all.set_index("Model")[metrics_1],
    annot=True,
    fmt=".3f",
    cmap=cmap,
    cbar=True,
    linewidths=0.5,
    linecolor="gray"
)
plt.title("Model Metrics Heatmap", fontsize=18, weight="bold")
plt.xticks(fontsize=12, rotation=45, ha="right")
plt.yticks(fontsize=12)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/project_first/heatmap.png", dpi=900)
plt.close()

```

```

[ ]: # prints the system's information used for the projects
def run_cmd(cmd):
    try:

```

```

        out = subprocess.check_output(cmd, shell=True, text=True)
        return out.strip()
    except Exception:
        return "N/A"

def get_full_system_info():
    info = {}

    info["Operating System"] = platform.platform()
    info["Python Version"] = platform.python_version()
    info["Architecture"] = platform.machine()

    cpu_info = get_cpu_info()
    info["CPU Brand"] = cpu_info.get("brand_raw", "N/A")
    info["CPU Architecture"] = cpu_info.get("arch", "N/A")
    info["CPU Cores (Logical)"] = psutil.cpu_count(logical=True)
    info["CPU Cores (Physical)"] = psutil.cpu_count(logical=False)

    freq = psutil.cpu_freq()
    if freq:
        info["CPU Frequency (Current MHz)"] = round(freq.current, 2)
        info["CPU Frequency (Max MHz)"] = round(freq.max, 2)
    else:
        info["CPU Frequency (Current MHz)"] = "N/A"

    info["CPU Usage (%)"] = psutil.cpu_percent(interval=1)

    mem = psutil.virtual_memory()
    info["RAM Total (GB)"] = round(mem.total / (1024**3), 2)
    info["RAM Available (GB)"] = round(mem.available / (1024**3), 2)
    info["RAM Used (GB)"] = round(mem.used / (1024**3), 2)
    info["RAM Usage (%)"] = mem.percent

    dmidecode = run_cmd("sudo dmidecode --type memory | grep 'Type:' | head -n1")
    info["RAM Type"] = dmidecode.replace("Type:", "").strip() if "Error" not in dmidecode else "N/A"

    try:

```

```

        disk = psutil.disk_usage('/')
        info["Storage Total (GB)"] = round(disk.total / (1024**3), 2)
        info["Storage Used (GB)"] = round(disk.used / (1024**3), 2)
        info["Storage Free (GB)"] = round(disk.free / (1024**3), 2)
        info["Storage Usage (%)"] = disk.percent
    except Exception:
        info["Storage Total (GB)"] = "N/A"

    storage_type = run_cmd("cat /sys/block/sda/queue/rotational") # 0=SSD, 1=HDD
    if storage_type == "0":
        info["Storage Type"] = "SSD / NVMe"
    elif storage_type == "1":
        info["Storage Type"] = "HDD"
    else:
        info["Storage Type"] = "Unknown"

    info["CPU Detailed Info (lscpu)"] = run_cmd("lscpu")

    return info

system_info = get_full_system_info()

df = pd.DataFrame(list(system_info.items()), columns=["Property", "Value"])
display(df)

print("\n===== CPU =====\n")
print(system_info["CPU Detailed Info (lscpu)"])

```

```

Property \
0      Operating System
1      Python Version
2      Architecture
3      CPU Brand
4      CPU Architecture
5      CPU Cores (Logical)
6      CPU Cores (Physical)
7  CPU Frequency (Current MHz)
8      CPU Frequency (Max MHz)
9      CPU Usage (%)
10     RAM Total (GB)
11     RAM Available (GB)
12     RAM Used (GB)

```

```

13          RAM Usage (%)
14          RAM Type
15      Storage Total (GB)
16      Storage Used (GB)
17      Storage Free (GB)
18      Storage Usage (%)
19      Storage Type
20  CPU Detailed Info (lscpu)

```

```

                                Value
0      Linux-6.6.105+-x86_64-with-glibc2.35
1                                3.12.12
2                                x86_64
3      Intel(R) Xeon(R) CPU @ 2.20GHz
4                                X86_64
5                                8
6                                4
7                                2200.16
8                                0.0
9                                0.8
10                               50.99
11                               48.79
12                               1.66
13                               4.3
14
15                               225.83
16                               39.44
17                               186.38
18                               17.5
19      SSD / NVMe
20  Architecture:                x86_6...

```

===== CPU =====

```

Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                46 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       8
On-line CPU(s) list:         0-7
Vendor ID:                    GenuineIntel
Model name:                   Intel(R) Xeon(R) CPU @ 2.20GHz
CPU family:                   6
Model:                        79
Thread(s) per core:           2
Core(s) per socket:           4
Socket(s):                     1

```

```

Stepping:                                0
BogoMIPS:                               4400.32
Flags:                                   fpu vme de pse tsc msr pae mce cx8 apic
sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx
pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology nonstop_tsc cpuid
tsc_known_freq pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe
popcnt aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs
ibpb stibp fsgsbase tsc_adjust bmi1 hle avx2 smep bmi2 erms invpcid rtm rdseed
adx smap xsaveopt arat md_clear arch_capabilities
Hypervisor vendor:                       KVM
Virtualization type:                     full
L1d cache:                              128 KiB (4 instances)
L1i cache:                              128 KiB (4 instances)
L2 cache:                               1 MiB (4 instances)
L3 cache:                               55 MiB (1 instance)
NUMA node(s):                            1
NUMA node0 CPU(s):                       0-7
Vulnerability Gather data sampling:       Not affected
Vulnerability Indirect target selection:  Vulnerable
Vulnerability Itlb multihit:              Not affected
Vulnerability L1tf:                       Mitigation; PTE Inversion
Vulnerability Mds:                        Vulnerable; SMT Host state unknown
Vulnerability Meltdown:                   Vulnerable
Vulnerability Mmio stale data:             Vulnerable
Vulnerability Reg file data sampling:      Not affected
Vulnerability Retbleed:                   Vulnerable
Vulnerability Spec rstack overflow:        Not affected
Vulnerability Spec store bypass:          Vulnerable
Vulnerability Spectre v1:                  Vulnerable: __user pointer sanitization
and usercopy barriers only; no swaps barriers
Vulnerability Spectre v2:                  Vulnerable; IBPB: disabled; STIBP:
disabled; PBRSE-eIBRS: Not affected; BHI: Vulnerable
Vulnerability Srbds:                       Not affected
Vulnerability Tsa:                         Not affected
Vulnerability Tsx async abort:             Vulnerable

```

```

[ ]: #saves and prints the libraries used in this project
packages = sorted([(d.project_name, d.version) for d in pkg_resources.
    ↪working_set])
df_libs = pd.DataFrame(packages, columns=["Library", "Version"])
print("=== Library Versions ===")
display(df_libs.head(20))
df_libs.to_excel("/content/drive/MyDrive/project_first/library_versions.xlsx",
    ↪index=False)

```

```

=== Library Versions ===

```

```

Library Version

```

0	Bottleneck	1.4.2
1	Brotli	1.1.0
2	Cython	3.0.12
3	Farama-Notifications	0.0.4
4	GDAL	3.8.4
5	PyDrive2	1.21.3
6	PyGObject	3.42.0
7	PyJWT	2.10.1
8	PySocks	1.7.1
9	Send2Trash	1.8.3
10	absl-py	1.4.0
11	absolufy-imports	0.3.1
12	accelerate	1.10.1
13	aiofiles	24.1.0
14	aiohappyeyeballs	2.6.1
15	aiohttp	3.13.0
16	aiosignal	1.4.0
17	alabaster	1.0.0
18	albucore	0.0.24
19	alumentations	2.0.8